

الجمهورية العربية السورية
المعهد العالي للعلوم التطبيقية والتكنولوجيا
قسم المعلوماتية
العام الدراسي 2025/2026

مشروع سنة خامسة

منظومة honeypot جاهزة للنشر مع نموذج ذكاء اصطناعي مُدمج لتصنيف نوع الهجوم آلياً

تقديم

حسين سلوم

إشراف

د. محمد عصورة

م. مفيد دوزكنجي

2026-2025

المحتويات

2	المحتويات
7	قائمة الاختصارات
9	قائمة المصطلحات
11	الجانب العملي من المشروع: منظومة honeypot جاهزة للنشر مع نموذج ذكاء اصطناعي مُدمج لتصنيف نوع الهجوم آلياً
11	لمحة عن العمل المنجز
12	المتطلبات الوظيفية وغير الوظيفية للمشروع
12	مقدمة
12	١. المتطلبات الوظيفية
12	١.١ التقاط الهجمات وتسجيل بياناتها
12	١.٢ توليد بيانات هجوم موسومة
12	١.٣ التصدير واستخراج السمات
13	١.٤ تدريب النماذج ومقارنتها واختيار الأفضل
13	١.٥ التصنيف الآلي
13	١.٦ التصنيف الحي
13	١.٧ العرض والتصور
13	١.٨ التحقق من سلامة البيانات
13	٢. المتطلبات غير الوظيفية
14	٢.١ الأداء (performance)
14	٢.٢ الموثوقية والمتانة (reliability & robustness)
14	٢.٣ الدقة وسلامة البيانات (accuracy & data integrity)
14	٢.٤ الأمان والعزل (security & isolation)
14	٢.٥ قابلية النقل (portability)
14	٢.٦ قابلية الصيانة والتوسع (maintainability & extensibility)
15	٢.٧ قابلية الاستخدام (usability)
15	٢.٨ كفاءة الموارد (resource efficiency)
15	٢.٩ التوثيق وإعادة الإنتاج (documentation & reproducibility)
16	تصميم النظام
16	مقدمة
16	مخطط حالات الاستخدام
16	مخطط الكتل العام
18	طوبولوجيا المختبر
18	مخطط المكونات
20	نموذج البيانات
20	مخطط التدفق الشامل
22	مخطط نموذج الذكاء الاصطناعي

22	مخطط تدفق التصنيف الحي
24	مخطط التتابع
25	التنفيذ والاختبارات
25	مقدمة
25	١. معمارية النظام المتبعة
25	١.١ النمط المعماري العام
25	١.٢ معمارية الالتقاط والتخزين
25	١.٣ معمارية توليد البيانات والوسم
25	١.٤ معمارية المعالجة (التصدير واستخراج السمات)
26	١.٥ معمارية التدريب والنموذج
26	١.٦ معمارية التصنيف الحي والعرض
26	١.٧ خلاصة القرارات المعمارية
26	٢. تفصيل أجزاء النظام وتكاملها
26	٢.١ أجزاء النظام
27	٢.٢ عقود التكامل بين الأجزاء
27	٣. الشيفرة الوصفية (Pseudocode)
27	٣.١ مشغل الحملة
28	٣.٢ الوسم التلقائي بالنافذة والمصدر
28	٣.٣ تصدير الأحداث
28	٣.٤ استخراج السمات
29	٣.٥ تدريب النماذج واختيار الأفضل
29	٣.٦ التصنيف الحي (كشف الجولات المحدود بالخمول)
31	١. بيئة العمل والبنية التحتية
31	١.١ الأجهزة والشبكة
32	١.٢ نشر T-Pot وتقليص الحاويات
33	١.٣ منظومة الرصد والتحليل
33	١.٤ الوصول إلى البيانات وتنقيتها
34	١.٥ الخدمات التي تحاكيها الـ honeypots والمستشعرات
35	٢. منهجية جمع البيانات
35	٢.١ تعريف العينة والوسم التلقائي
35	٢.٢ النصوص البرمجية للحملة وتصدير البيانات
36	٢.٣ حجم البيانات النهائي
36	٣. حملات الهجوم بأنواعها الخمسة
36	٣.١ المسح (scan)
37	٣.٢ التخمين عبر الإنترنت (online guessing)
39	٣.٣ ما بعد الاختراق (post-exploitation)
41	٣.٤ هجوم الويب (web attack)

42	 ٣.٥ حجب الخدمة (denial of service / DoS)
43	 ٤. معالجة جودة البيانات
43	 ٤.١ فساد عيّنات ما بعد الاختراق (P016–P030)
44	 ٤.٢ إعادة جمع فئة التخمين عبر الإنترنت
44	 ٤.٣ تصحيح عدّ محاولات الدخول (n_auth_other)
44	 ٤.٤ التعامل مع التسجيل المتأخّر في حجب الخدمة
44	 ٥. استخراج السمات وبناء مجموعتي البيانات
45	 ٥.١ فلسفة السمات: السلوك لا المستشعر
45	 ٥.٢ السمات السلوكية الاثنان والثلاثون
46	 ٥.٣ ضمانات منع تسريب المعلومات (leakage safeguards)
46	 ٦. تدريب النماذج ومقارنتها
46	 ٦.١ بروتوكول التقييم
47	 ٦.٢ إثبات الفرضية: السلوك لا المستشعر
48	 ٦.٣ ضبط الغاية العشوائية
48	 ٦.٤ مقارنة ستّة نماذج
50	 ٦.٥ مجموعة الأخطاء الموحدة
51	 ٧. نشر النموذج والتصنيف الحي
51	 ٧.١ النموذج المنشور والملفات المحفوظة
52	 ٧.٢ معمارية التصنيف والربط بـ Kibana
52	 ٧.٣ التصنيف الحي: كشف الجولات المحدود بالخمّل
53	 ٨. لوحة معلومات Kibana الحية لعرض التصنيف
53	 ٨.١ الهدف ومسار البيانات والقرار المعماري
54	 ٨.٢ اختبار إمكانية الكتابة إلى Elasticsearch
54	 ٨.٣ تعديل النصّ البرمجي live_runs.py لكتابة التنبؤات
55	 ٨.٤ التحقق الشامل من طرف إلى طرف
57	 ٨.٥ تسجيل عرض البيانات في Kibana
57	 ٨.٦ بناء لوحة المعلومات
58	 9. خطة الاختبارات
58	 9.1 اختبارات الوحدة (unit tests)
59	 ٤.٢ اختبارات التكامل (integration tests)
59	 ٤.٣ اختبارات الأمان (security tests)
60	 ٤.٤ اختبار التحميل (load test)
60	 ٩. تحليل المهاجم: المصدر وطريقة الدخول والهدف
61	 ٩.١ من هو المصدر
62	 ٩.٢ كيف تمّ الدخول
62	 ٩.٣ ما هدف المهاجم
63	 ٩.٤ ربط الأسئلة الثلاثة بسمات النموذج

١٠. ملاحظات على قصور الـ honeypots	63
خلاصة	64

قائمة الأشكال

- الشكل 1: مخطّط حالات الاستخدام. 16
- الشكل 2: مخطّط الكتل العام للنظام. 17
- الشكل 3: طوبولوجيا بيئة الاختبار المعزولة. 18
- الشكل 4: مخطّط المكونات والوحدات البرمجية. 19
- الشكل 5: نموذج البيانات (مخطّط الكيانات والعلاقات). 20
- الشكل 6: مخطّط تدفق الخطّ الكامل (البناء والتشغيل الحي). 21
- الشكل 7: مخطّط نموذج الذكاء الاصطناعي (مصنّف XGBoost). 22
- الشكل 8: مخطّط تدفق التصنيف الحي (كشف الجولات المحدود بالخمول). 23
- الشكل 9: مخطّط التتابع للتصنيف الحي. 24
- الشكل 10: مخطّط بيئة الاختبار المعزولة يوضّح آلة الـ honeypot وآلة Kali و topology الشبكة. 31
- الشكل 11: خرج الأمر dps (docker ps) يُظهر الحاويات العشرين العاملة بعد التقليل وحالتها. 33
- الشكل 12: نتيجة أمر nmap تُظهر المنافذ المفتوحة واللائقات (banners) الوهمية على آلة T-Pot. 37
- الشكل 13: خرج أداة Hydra في جولة SSH يُظهر سلسلة المحاولات ثم الزوج الناجح. 38
- الشكل 14: كلمات المرور (password) التي جرّبها المهاجم حيث سجّلها Cowrie. 38
- الشكل 15: أسماء المستخدمين (username) التي جرّبها المهاجم كما سجّلها Cowrie. 39
- الشكل 16: حدث cowrie.login.success يُظهر الزوج الناجح root/qwerty. 39
- الشكل 17: جلسة SSH لما بعد الاختراق داخل Cowrie تُظهر تسلسل أوامر الاستطلاع. 40
- الشكل 18: لوحة Kibana: العنوان المصدري، وأكثر أوامر cowrie.command.input. 41
- الشكل 19: خرج نصّ التحقق verify_DOS.py يُظهر عدد الأحداث الملتقطة لكلّ جولة من جولات حجب الخدمة. 41
- الشكل 20: مصفوفة الالتباس للعشوائية التي تظهر نفس الاخطاء لنموذج مدرب على كامل السمات او على السلوكية فقط. 43
- الشكل 21: مخطّط t-SNE يُظهر تجمّع الفئات الخمس وانفصالها في فضاء السمات. 47
- الشكل 22: الرسم البياني لمقارنة النماذج الستة بحسب الدقّة. 48
- الشكل 23: أهمية السمات (feature importance) في نموذج XGBoost المنشور. 49
- الشكل 24: مصفوفة الالتباس في نموذج XGBoost المنشور. 50
- الشكل 25: خرج live_runs.py وهو يصنّف جولة حجب خدمة حيّة مع درجات الثقة والسمات الدالة. 51
- الشكل 26: مخطّط المعمارية: live_runs.py يصنّف الجولة ويكتبها في فهرس ml-predictions، ثم تقرأه لوحة Kibana وترسمه. 53
- الشكل 27: خرج طرفية المصنّف الحيّ يُظهر تصنيف المسح. 54
- الشكل 28: أوّل ظهورٍ للتنبؤ في لوحة Kibana بعد التحقق الشامل: جولة مسح واحدة في الخطّ الزمني وتوزيع الفئات. 56
- الشكل 29: الجزء السفلي من لوحة kibana يعرض المعلومات التي تم تسجيلها عن عملية المسح. 56
- الشكل 30: لوحة المعلومات (الجزء الأعلى): أنواع الهجوم عبر الزمن وتوزيع الفئات. 57
- الشكل 31: لوحة المعلومات (الجزء الأسفل): متوسط الثقة وجدول آخر الاكتشافات. 58
- الشكل 32: حقل os.keyword في بيانات p0f يُظهر بصمة نظام تشغيل المصدر (Linux 2.2.x-3.x). 61
- الشكل 33: حقل src_ip يُظهر العنوان المصدري 192.168.32.1 في كلّ الأحداث. 61
- الشكل 34: أحداث p0f في Kibana بحقولها @timestamp و dist و src_ip و os. 61
- الشكل 35: تسجيل Cowrie للعنوان المصدري 192.168.32.1. 62
- الشكل 36: حدث cowrie.login.success يُظهر الزوج الناجح root/qwerty. 62
- الشكل 37: تسلسل أوامر cowrie.command.input داخل إحدى جلسات ما بعد الاختراق. 63

قائمة الاختصارات

الجدول 1: قائمة الاختصارات المستعملة في التقرير.

الترجمة العربية	Full English term	Abbreviation
علم الإقرار في بروتوكول TCP	Acknowledgement (TCP flag)	ACK
واجهة برمجة التطبيقات	Application Programming Interface	API
قيم مفصولة بفواصل (صيغة ملفّ جدولي)	Comma-Separated Values	CSV
حجب الخدمة	Denial of Service	DoS
مكدس ELK للتخزين والتجميع والعرض	Elasticsearch, Logstash, Kibana	ELK
مخطّط الكيانات والعلاقات	Entity–Relationship Diagram	ERD
محرك التخزين والبحث Elasticsearch	Elasticsearch	ES
علم إنهاء الاتصال في بروتوكول TCP	Finish (TCP flag)	FIN
بروتوكول نقل الملفات	File Transfer Protocol	FTP
بروتوكول نقل النصّ التشعبي	Hypertext Transfer Protocol	HTTP
بروتوكول رسائل التحكم في الإنترنت	Internet Control Message Protocol	ICMP
أنظمة التحكم الصناعي	Industrial Control Systems	ICS
نظام كشف الاختراق	Intrusion Detection System	IDS
العنوان المنطقي في بروتوكول الإنترنت	Internet Protocol (address)	IP
صيغة تبادل بيانات نصية	JavaScript Object Notation	JSON
خوارزمية الجيران الأقرب	K-Nearest Neighbours	KNN
الطبقة السابعة (طبقة التطبيقات)	Layer 7 (application layer)	L7
إدراج الملفات المحلية	Local File Inclusion	LFI
متوسط مقياس F1 على الفئات	Macro-averaged F1 score	macro-F1
الشبكة العصبية متعددة الطبقات	Multi-Layer Perceptron	MLP
ترجمة عناوين الشبكة	Network Address Translation	NAT
تحليل المكونات الرئيسية	Principal Component Analysis	PCA
نواة الدالة القاعدية الشعاعية	Radial Basis Function (SVM kernel)	RBF
بروتوكول سطح المكتب البعيد	Remote Desktop Protocol	RDP
علم إعادة تعيين الاتصال في بروتوكول TCP	Reset (TCP flag)	RST
أنظمة التحكم الإشرافي وجمع البيانات	Supervisory Control and Data Acquisition	SCADA

الترجمة العربية	Full English term	Abbreviation
بروتوكول مشاركة الملفات SMB	Server Message Block	SMB
لغة الاستعلام البنيوية	Structured Query Language	SQL
حقن لغة الاستعلام البنيوية	SQL Injection	SQLi
الصدفة الآمنة للاتصال عن بُعد	Secure Shell	SSH
آلة المتجهات الداعمة	Support Vector Machine	SVM
علم بدء المزامنة في بروتوكول TCP	Synchronise (TCP flag)	SYN
بروتوكول التحكم بالنقل	Transmission Control Protocol	TCP
أمان طبقة النقل	Transport Layer Security	TLS
طريقة تخفيض الأبعاد t-SNE	t-distributed Stochastic Neighbour Embedding	t-SNE
بروتوكول مخططات بيانات المستخدم	User Datagram Protocol	UDP
التوقيت العالمي المنسق	Coordinated Universal Time	UTC
الحوسبة الشبكية الافتراضية	Virtual Network Computing	VNC
نظام Windows الفرعي لـ linux، الإصدار الثاني	Windows Subsystem for Linux, version 2	WSL2
التعزيز المتدرج المتطرف (نموذج شجري)	eXtreme Gradient Boosting	XGBoost
مسح شجرة الميلاد بأعلام FIN و PSH و URG	Christmas-tree scan (FIN+PSH+URG flags)	XMAS
البرمجة عبر المواقع	Cross-Site Scripting	XSS

قائمة المصطلحات

الجدول 2: قائمة المصطلحات التقنية ومقابلاتها العربية.

الترجمة العربية	English term
مصيدة رقمية تحاكي خدمة حقيقية لجذب المهاجم وتسجيل سلوكه	honeypot
التلمترية: البيانات التي تسجلها المنظومة عن كل حدث	telemetry
مستشعر يراقب حركة الشبكة دون أن يقدم خدمة	sensor
حاوية برمجية معزولة تُدار عبر Docker	container
جولة هجوم واحدة، وهي العينة الواحدة في مجموعة البيانات	attack run
سجل التشغيل الذي يوثق كل جولة بنافذتها الزمنية	run ledger
فترة تهدئة بين جولتين تمنع تداخل النوافذ الزمنية	cooldown
الوسم: نسب كل حدث إلى جولته وفتتها	labeling
سمة: قيمة رقمية تصف جانباً من سلوك الجولة	feature
متجه السمات الذي يمثل الجولة أمام النموذج	feature vector
مجموعة البيانات المستعملة في التدريب والتقييم	dataset
التعلم الآلي	machine learning
التصنيف: إسناد الجولة إلى إحدى الفئات الخمس	classification
المصنّف: النموذج الذي يُنتج التنبؤ	classifier
التحقق المتقاطع: تقييم على طيات متعددة بدل اقتطاع مجموعة اختبار	cross-validation
التقسيم الطبقي إلى طيات يحفظ توازن الفئات	stratified k-fold
الدقة: نسبة العينات المصنّفة تصنيفاً صحيحاً	accuracy
مصفوفة الالتباس التي تُظهر أين يخطأ النموذج بين الفئات	confusion matrix
أهمية السمات: مقدار اعتماد النموذج على كل سمة	feature importance
مُعامل فائق يُضبط قبل التدريب	hyperparameter
البحث الشبكي عن أفضل تركيبة مُعاملات	grid search
التحجيم: توحيد مقاييس السمات قبل التدريب	scaling
المبالغة في الملاءمة: حفظ النموذج للبيانات بدل تعلم القاعدة	overfitting
تسريب المعلومات: اعتماد النموذج على حقل يكشف الإجابة	leakage
تخفيض الأبعاد لعرض البيانات بصرياً	dimensionality reduction
خطّ المعالجة: سلسلة المراحل من توليد الهجوم إلى التنبؤ	pipeline
عقد بيانات: الشكل المتفق عليه للبيانات بين مرحلتين	data contract
عديم الأثر: لا يُكرّر عملاً أنجز سابقاً عند إعادة التشغيل	idempotent
إعادة الإنتاج: إمكانية الحصول على النتائج نفسها مجدداً	reproducibility
قابلية النقل بين البيئات والإصدارات	portability

الترجمة العربية	English term
التدهور اللطيف: استمرار الوظيفة الأساسية عند فشل جزء ثانوي	graceful degradation
تماثل السمات بين التدريب والتشغيل الحي	feature parity
كشف الجولات المحدود بالخمول: إنهاء الجولة عند سكون النشاط لمدة idle	idle-bounded run detection
آلة حالات تنتقل بين حالات محدّدة وفق الأحداث	state machine
الاستطلاع الدوري للأحداث الجديدة	polling
مؤقت زمني يوقف الأداة عند تجاوز حد معيّن	timeout
فهرس Elasticsearch الذي تُخزّن فيه الوثائق	index
وثيقة: السجلّ الواحد داخل فهرس Elasticsearch	document
التخطيط: تعريف أنواع حقول الفهرس	mapping
عرض بيانات في Kibana يربط اللوحة بفهرس معيّن	data view
لوحة معلومات تعرض النتائج بصرياً	dashboard
واجهة Elasticsearch لسحب النتائج الكبيرة على صفحات	Scroll API
حمولة: المحتوى الذي يرسله المهاجم لاستغلال الخدمة	payload
لافتة: نصّ التعريف الذي تردّ به الخدمة على الاتصال	banner
مصافحة: تبادل بدء الاتصال بين الطرفين	handshake
تدفّق: اتصال شبكيّ يرصده Suricata	flow
توقيع: قاعدة كشف تطابق نمطاً معروفاً	signature
التمكين والبقاء: محاولة المهاجم ضمان عودته إلى النظام	persistence
نصّ برمجيّ	script
النشر: تشغيل النموذج في بيئة العمل الفعلية	deployment

الجانب العملي من المشروع: منظومة honeypot جاهزة للنشر مع نموذج ذكاء اصطناعي مُدمج لتصنيف نوع الهجوم آلياً

لمحة عن العمل المنجز

يتناول هذا الجزء الشقّ التطبيقي من المشروع، أي ما جرى بناؤه وتطبيقه وتدريبه واختباره فعلياً. باختصار، أنشأنا مختبراً معزولاً يقوم على منظومة T-Pot للـ honeypot، ثم نفّذنا عليه حملات هجوم واقعية من خمسة أنواع مختلفة، وجمعنا ما سجّلته المنظومة من بيانات (telemetry) عبر مكّس ELK، ثم استخرجنا من هذه البيانات مجموعة من السمات (features) الوصفية لسلوك كل هجوم، ودربنا عليها عدّة نماذج تعلّم آلة وقارناً بينها، ثم نشرنا النموذج الأفضل وربطناه بمُصنّف يعمل بشكل حيّ (live) على البيانات الواردة، وأخيراً بنينا لوحة معلومات (dashboard) في Kibana تعرض هذه التصنيفات بشكل حيّ وتُخزّنها.

والفكرة المحورية للمشروع هي بناء honeypot جاهز للنشر والتوظيف، مدمج به نموذج ذكاء اصطناعي قادر على تصنيف نوع الهجوم آلياً — أي منظومة واحدة متكاملة تلتقط الهجوم وتصنّفه في آنٍ معاً، لا مجرد نموذج تصنيف معزول. وقد بلغ أفضل النماذج (XGBoost) دقّةً في التحقق المتقاطع (cross-validation) قاربت 98.7% على خمس فئات متوازنة. ومن المبادئ التصميمية التي تحقّقنا منها لاحقاً أنّ هذا التصنيف يقوم على سلوك الهجوم لا على معرفة أيّ honeypot التقطه — أي أنّ ما يفصل بين المسح (scan) والتخمين عبر الإنترنت (online guessing) وما بعد الاختراق (post-exploitation) وهجوم الويب (web attack) وحجب الخدمة (DoS) هو طريقة تصرّف الهجوم في الشبكة، لا هويّة المستشعر (sensor) الذي رصده — وهو ما يجعل المنظومة قابلةً للتعميم. يعرض هذا الجزء الخطوات كاملةً بالترتيب: بيئة العمل والبنية التحتية، ثم منهجية جمع البيانات، ثم حملات الهجوم بأنواعها الخمسة، ثم معالجة جودة البيانات، ثم استخراج السمات وبناء مجموعات البيانات، ثم تدريب النماذج ومقارنتها، ثم نشر النموذج والتصنيف الحيّ، ثم بناء لوحة معلومات Kibana الحيّة، ثم تحليل المهاجم (من هو، وكيف دخل، وماذا يريد)، وأخيراً أهمّ ما لاحظناه من قصور في أدوات الـ honeypot نفسها.

المتطلبات الوظيفية وغير الوظيفية للمشروع

مقدمة

يُعنى هذا الجزء بتحديد متطلبات المنظومة التي بُنيت في المشروع، وهي منظومة honeypot جاهزة للنشر والتوظيف مدمجٌ بها نموذج ذكاء اصطناعي يصنّف نوع الهجوم آلياً. ونقسم المتطلبات إلى قسمين: **المتطلبات الوظيفية (functional requirements)** التي تصف ما ينبغي أن تفعله المنظومة من وظائف وخدمات، و**المتطلبات غير الوظيفية (non-functional requirements)** التي تصف خصائص الجودة التي ينبغي أن تتوفر فيها كالإداء والموثوقية والأمان وقابلية النقل.

١. المتطلبات الوظيفية

تصف المتطلبات الوظيفية الوظائف التي تؤديها المنظومة، من التقاط الهجوم في المختبر إلى تصنيفه وعرضه حياً. ونعرضها مجموعة مجموعة بحسب مرحلة العمل.

١.١ التقاط الهجمات وتسجيل بياناتها

الوظيفة الأساسية للمنظومة أن تجذب المهاجمين وتسجل سلوكهم بدقة. لذلك يجب على المنظومة أن:

1. توفر بيئة honeypot متعددة تحاكي خدماتٍ مُعرّضة للهجوم (SSH و Telnet و FTP و VNC و RDP وخادم ويب وموقع WordPress ومصيدة عامّة)، بحيث يتفاعل معها المهاجم ظناً منه أنّها أنظمة حقيقية.
2. تسجل كلّ حدثٍ تفاعليّ (event) بتفاصيله الكاملة في مخزنٍ مركزيّ للبحث (Elasticsearch).
3. تراقب النشاط على مستوى الشبكة عبر مستشعرات (Suricata و p0f و fatt) مستقلة عن الـ honeypots، لتوفير رؤية إضافية للاتصالات والبصمات لا تعتمد على أيّ مصيدةٍ بعينها.

١.٢ توليد بيانات هجوم موسومة

لتدريب النموذج تحتاج المنظومة إلى بياناتٍ حقيقية موسومة، لا إلى بياناتٍ مصنّعة. لذلك يجب على المنظومة أن:

1. تولّد حملات هجوم من خمس فئات: المسح (scan)، والتخمين عبر الإنترنت (online guessing)، وما بعد الاختراق (post-exploitation)، وهجوم الويب (web attack)، وحجب الخدمة (DoS).
2. تسجل كلّ جولة هجوم (attack run) بوصفها عيّنةً واحدةً موسومة في سجلّ تشغيل (run ledger) يحوي مُعرّفها ووسمها وأداتها وأمرها الفعليّ ونافذتها الزمنية وهدفها ومصدرها.
3. تسم الأحداث الملتقطة وسمّاً تلقائياً بمطابقة النافذة الزمنية (UTC) والعنوان المصدريّ للهجوم، دون الحاجة إلى وسيم يدويّ.
4. تجعل نصوص الحملات البرمجية (scripts) قابلةً للاستئناف بعد التوقّف، ومحدودةً بمؤقت زمنيّ (timeout) يضمن توقّفها النظيف.

١.٣ التصدير واستخراج السمات

يجب على المنظومة أن تحوّل البيانات الخام إلى صورة رقمية قابلة للتعلّم. لذلك يجب أن:

1. تصدر أحداث كلّ جولة من Elasticsearch إلى ملفات منفصلة، مُرشَّحة بالنافذة الزمنية والعنوان المصدري.
2. تستخرج من كلّ جولة متَّجهاً رقمياً من السمات السلوكية (features) عددها اثنتان وثلاثون سمة، وتبني منها مجموعات بيانات التدريب.
3. تستبعد الحقول التي قد تسرّب الإجابة أو تخصّ البيئة لا الهجوم (العناوين، وأرقام المنافذ المصدريّة، ومعرّفات الجلسات، ووقت اليوم، وحقل التصنيف القائم على القواعد).

١.٤ تدريب النماذج ومقارنتها واختيار الأفضل

- يجب على المنظومة أن تبني المصنّف وتختار أفضله بموضوعية. لذلك يجب أن:
1. تدرب عدّة نماذج تعلّم آلة وتقارن بينها ضمن إطار تقييم موحّد بالتحقق المتقاطع الطبقيّ (stratified cross-validation).
 2. تعرض لكلّ نموذج مقاييسه: الدقّة (accuracy)، و macro-F1، ومصفوفة الالتباس (confusion matrix)، والعينات المخطئة.
 3. تختار النموذج الأفضل وتحفظه مع بياناته الوصفية (ترتيب السمات، وأسماء الفئات، والمعاملات، وإصدارات المكتبات) بصيغة قابلة لإعادة التحميل.

١.٥ التصنيف الآلي

- يجب أن تصنّف المنظومة أيّ هجوم مُقدّم (على هيئة متّجه سمات) إلى إحدى الفئات الخمس، وأن تُخرج الفئة المتنبّأ بها مصحوبةً بدرجة ثقة (confidence) وباحتمالات كلّ الفئات، لا بالفئة وحدها.

١.٦ التصنيف الحيّ

- يجب على المنظومة أن تعمل حيّاً على البيانات الواردة لحظة بلحظة. لذلك يجب أن:
1. تكتشف جولة هجوم كاملة في الزمن الحقيقيّ بأسلوب كشف الجولات المحدود بالخممول (idle-bounded run detection)، وتصنّفها بعد اكتمالها بوقت قصير (ما يقارب الدقيقة).
 2. تكتب كلّ تنبؤ حيّ وثيقّة (document) في فهرس مخصّص (ml-predictions) كي يُعرض ويُحفظ.
 3. تستمرّ في التصنيف حتى لو تعدّرت الكتابة إلى المخزن، فلا يتوقّف المصنّف بسبب فشل في طبقة العرض.

١.٧ العرض والتصوّر

- يجب أن توفّر المنظومة لوحة معلومات (dashboard) حيّة تعرض أنواع الهجوم عبر الزمن، وتوزيع الفئات، ومتوسط الثقة، وجدولاً بأخر الاكتشافات؛ وأن تتحدّث هذه اللوحة تلقائياً مع كلّ هجوم جديد.

١.٨ التحقق من سلامة البيانات

- يجب أن تتحقّق المنظومة من أنّ كلّ جولة قد التقطت عدداً غير صفريّ من الأحداث، ومن خلّو البيانات من التكرار ومن تداخل النوافذ الزمنية بين الجولات.

٢. المتطلبات غير الوظيفية

تصف المتطلبات غير الوظيفية خصائص الجودة التي ينبغي أن تتّصف بها المنظومة، وهي التي تفصل بين نظام «يعمل» ونظام «صالح للنشر والاعتماد عليه».

٢.١ الأداء (performance)

1. يجب أن يكتمل تصنيف الجولة الحيّة خلال ثوانٍ قليلة من انتهاء الهجوم، وأن تتحدّث اللوحة خلال نحو عشر ثوانٍ.
2. يجب أن يبلغ النموذج المنشور دقّةً عالية، وقد قاربت ٩٨,٧٪ في التحقّق المتقاطع على خمس فئاتٍ متوازنة.
3. يجب أن يتعامل خطّ التصدير واستخراج السمات مع حجم بيانات كبير (نحو ١,٤ مليون حدث) دون إخفاق.

٢.٢ الموثوقية والمتانة (reliability & robustness)

1. يجب ألا ينهار المصنّف الحيّ عند تعدّد الوصول إلى المخزن، بل يتدهور بلطف (graceful degradation) مكتفياً بطباعة تحذير.
2. يجب أن تكون نصوص الحملات البرمجية قابلةً للاستئناف بعد أيّ انقطاع دون فقدان ما أنجز.
3. يجب أن تكون عملية جمع البيانات قابلةً لإعادة الإنتاج (reproducible) بالكامل انطلاقاً من السجلّ.

٢.٣ الدقّة وسلامة البيانات (accuracy & data integrity)

1. يجب أن يكون الوسم قاطعاً لا لبس فيه، بلا نوافذ زمنية متداخلة، ومُتحقّقاً منه.
2. يجب أن تكون السمات خاليةً من تسريب المعلومات (leakage-safe) كي لا «يغشّ» النموذج.
3. يجب أن يكون التقييم أميناً، عبر تحقّق مقاطع على بياناتٍ نظيفة ومتوازنة لا على أرقامٍ متضخّمة.

٢.٤ الأمان والعزل (security & isolation)

1. يجب أن يعمل مختبر الـ honeypot في بيئةٍ معزولة تماماً عن الشبكات الحقيقية، فلا تتسرّب حركة الهجوم خارجها.
2. يجب أن يكون Elasticsearch مقيّداً بالعنوان المحلي، ولا يُوصَل إليه إلّا عبر نفق SSH (SSH tunnel) آمن.
3. يجب ألا تُرْفَع أيّ أسرارٍ أو معلوماتٍ حسّاسة إلى المستودع، فتُستبعد ملفات المفاتيح والاعتمادات (credentials).

٢.٥ قابلية النقل (portability)

1. يجب أن يُحفظ النموذج المنشور بصيغةٍ قابلة للنقل (صيغة XGBoost الأصلية) تعمل عبر مختلف إصدارات المكتبات والأجهزة.
2. يجب أن يعمل خطّ المشروع على بيئةٍ موثّقة ومحدّدة: Python و Anaconda على Windows لأعمال التعلّم، و Kali لتوليد الهجوم.

٢.٦ قابلية الصيانة والتوسّع (maintainability & extensibility)

1. يجب أن يكون الخطّ معيارياً (modular) بنصوصٍ برمجية منفصلة لكلّ مرحلة، وقابلاً لإعادة التشغيل كلّما نمت البيانات.

2. يجب أن يسمح بإضافة فئات هجوم أو سمات جديدة بتوسيع نصوص الحملات ومستخرج السمات، دون إعادة بناء جذرية.

3. يجب أن تُدار الشيفرة والمخرجات بنظام تحكم بالإصدارات (Git) ببنية واضحة وتوثيق مرافق.

٢.٧ قابلية الاستخدام (usability)

1. يجب أن تعرض اللوحة النتائج بوضوح يسهل قراءته لمحلل غير متخصص، بألوان ثابتة لكل فئة ولوحات مرتبة.

2. يجب أن تكون المخرجات مقروءة للبشر: أسماء فئات صريحة، ونسب مئوية بدل الأرقام الخام.

٢.٨ كفاءة الموارد (resource efficiency)

يجب أن تُقلص منظومة الـ honeypot لتناسب عتاداً متواضعاً (نحو ١٩ حاوية بدل نحو ٣٩)، مع إبقاء ما يلزم للفئات المستهدفة وللتنفيذ فقط، تخفيفاً لاستهلاك الذاكرة والمعالجة.

٢.٩ التوثيق وإعادة الإنتاج (documentation & reproducibility)

يجب أن يُوثق كل جولة وقرار ومُخرَج (سجل التشغيل، وخريطة الحقول، ودفاتر Jupyter، وملف README)، بحيث يستطيع أي شخص إعادة إنتاج الخط كاملاً والتحقق من نتائجه.

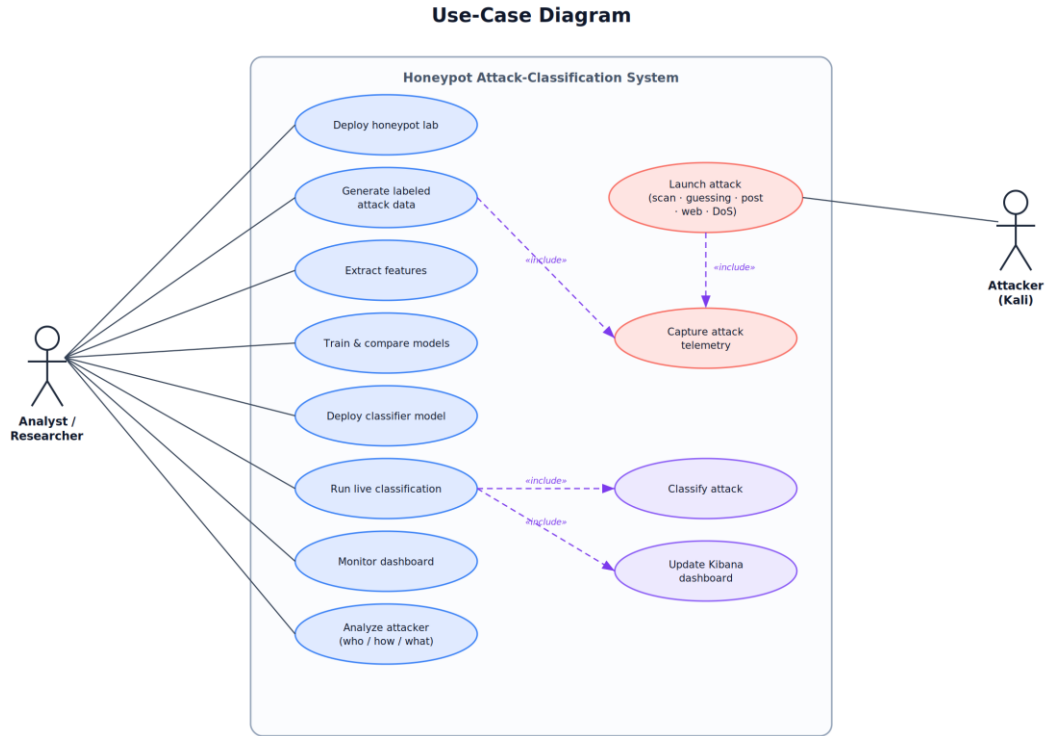
تصميم النظام

مقدمة

يعرض هذا الفصل تصميم النظام قبل الدخول في تفاصيل تنفيذه، وينتقل فيه العرض من الرؤية العامة إلى التفاصيل الدقيقة: من الجهات المتعاملة مع النظام ووظائفها، إلى البنية العامة، فالمكونات البرمجية، فتمودج البيانات، ثم الخطّ الكامل لمعالجة البيانات، وصولاً إلى تصميم نموذج الذكاء الاصطناعي وآلية التصنيف الحي. ولكلّ منظورٍ تصميميٍّ مخطّطٌ يوضّحه ويليه شرحٌ له، بحيث تتكامل المخطّطات في تكوين صورة واحدة عن بنية النظام وسلوكه.

مخطّط حالات الاستخدام

يبدأ التصميم من تحديد من يستعمل النظام وما الذي يفعله كلّ منهم، وهو ما يوضّحه مخطّط حالات الاستخدام (use-case diagram) في الشكل 1.



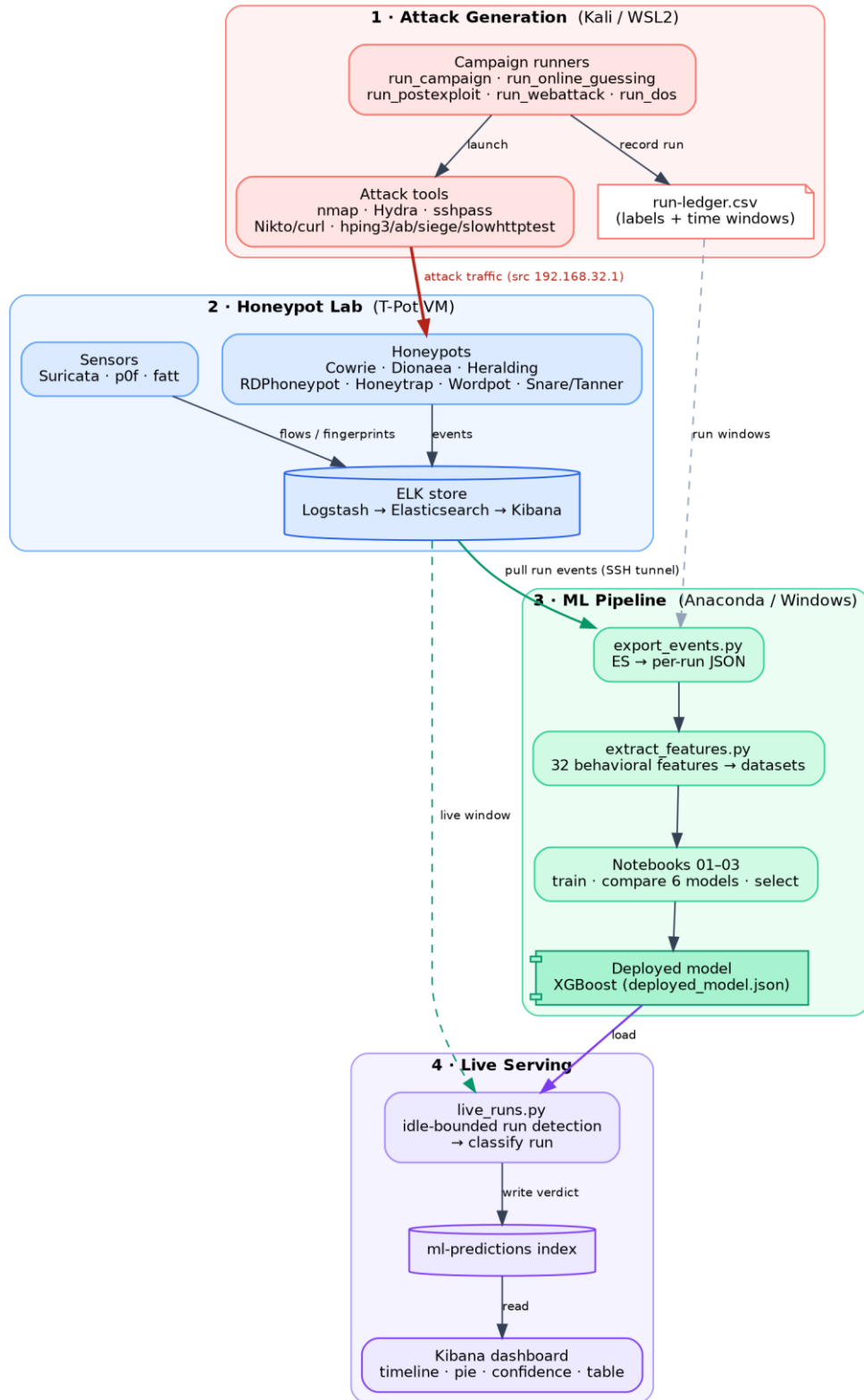
الشكل 1: مخطّط حالات الاستخدام.

لنظام جهتان متعاملتان (actors): الباحث (Analyst) والمهاجم (Attacker). يقوم الباحث بالوظائف الأساسية كلّها: نشر مختبر الـ honeypot، وتوليد بيانات هجوم موسومة، واستخراج السمات، وتدريب النماذج ومقارنتها، ونشر النموذج، وتشغيل التصنيف الحي، ومتابعة اللوحة، وتحليل المهاجم. أمّا المهاجم فوظيفته الوحيدة إطلاق الهجوم. وتوضّح علاقات التضمين (include) أنّ إطلاق الهجوم — وكذلك توليد البيانات — يتضمّن النقاط التلمزية، وأنّ تشغيل التصنيف الحي يتضمّن تصنيف الهجوم وتحديث اللوحة. وبهذا يربط المخطّط المتطلّبات الوظيفية بتعاملات ملموسة بين الجهات والنظام.

مخطّط الكتل العام

بعد تحديد الوظائف، تُعرّض البنية العامة للنظام مقسّمةً إلى مراحل، كما في الشكل 2.

Honeypot Attack-Classification System — Block Diagram



الشكل 2: مخطط الكتل العام للنظام.

يتكون النظام من أربع مراحل متتابعة: توليد الهجوم على Kali، ثم مختبر الـ honeypot مع منظومة ELK، ثم خط التعلم الآلي على Anaconda، وأخيراً التشغيل الحي وعرض النتائج في Kibana. وترتبط بين المراحل تدفقات واضحة: حركة الهجوم التي تصل بالعنوان 192.168.32.1، والتلمتيرية التي تُسحب عبر نفق SSH، والنموذج

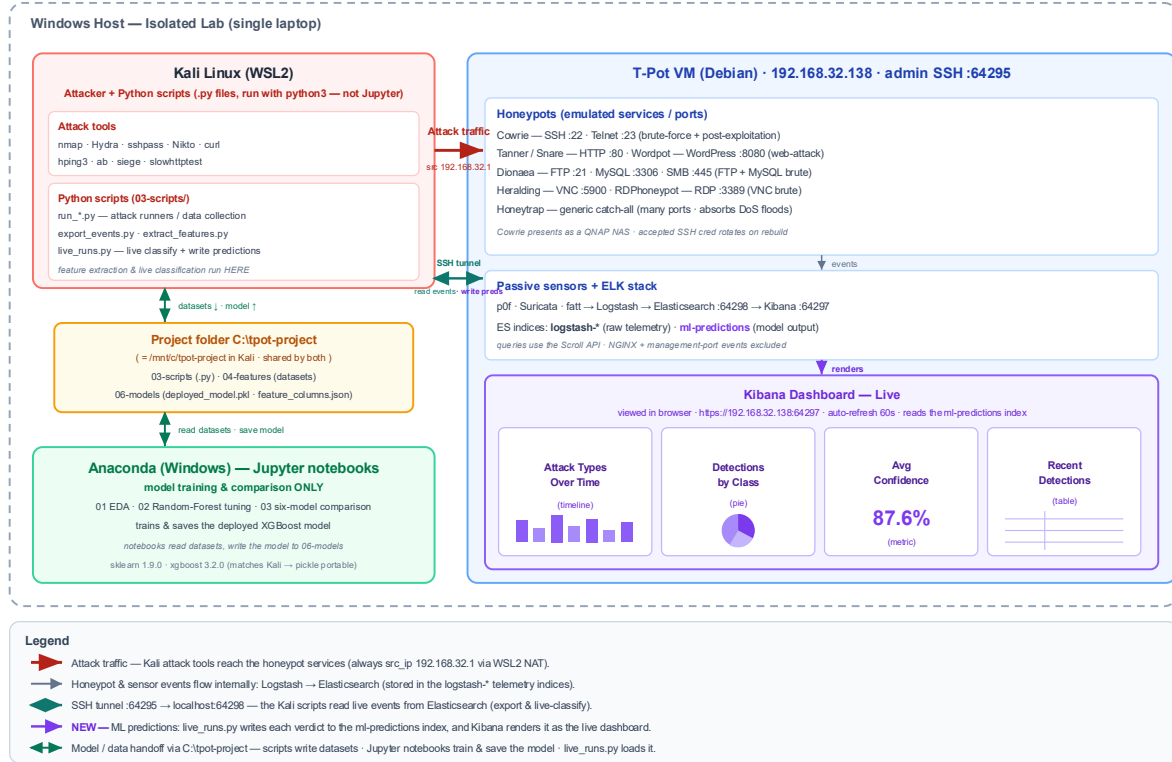
المنشور الذي يُحمّل في المصنّف الحي. وميزة هذا الفصل أنّه يجعل توليد البيانات وتخزينها والتعلّم منها وتشغيلها مراحل مستقلة، يمكن تشغيل كلّ منها والتفكير فيها على حدة.

طوبولوجيا المختبر

يوضّح الشكل 3 طوبولوجيا بيئة الاختبار.

Lab Topology — T-Pot Honeypot + ML Attack Classification

Isolated single-laptop lab: attack → collect telemetry → train model (Jupyter) → live-classify (script) → write predictions → Kibana dashboard



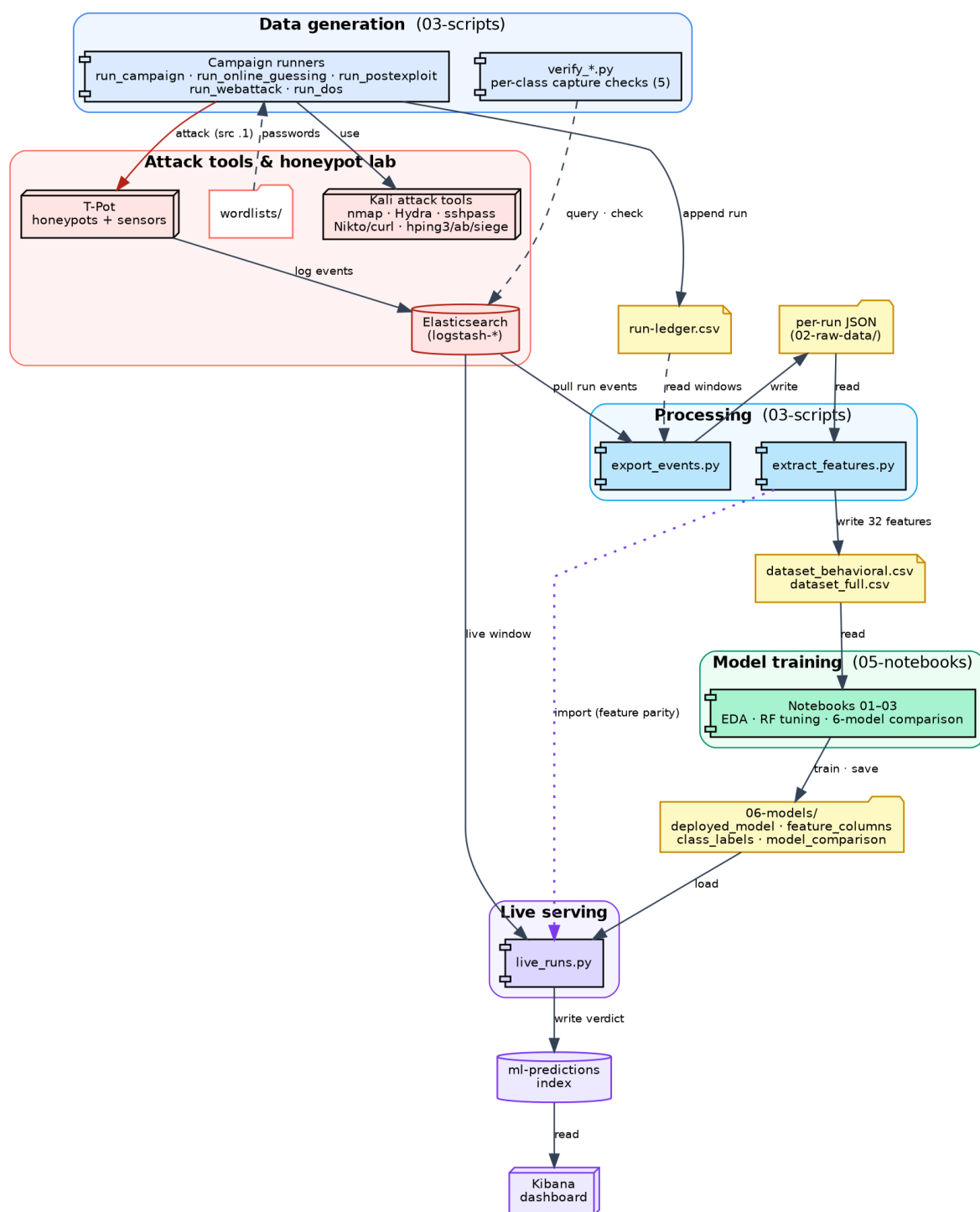
الشكل 3: طوبولوجيا بيئة الاختبار المعزولة.

يعمل المختبر بأكمله على جهاز واحد ضمن شبكة معزولة: آلة المهاجم Kali تعمل ضمن WSL2 على Windows، وآلة T-Pot بالعنوان 192.168.32.138، ومنظومة ELK للتخزين والعرض. ومن أهم ما يبرزه المخطط أنّ ترجمة عناوين WSL2 (NAT) تجعل كلّ حركة المهاجم تصل بالعنوان المصدري الثابت 192.168.32.1. ويحقّق هذا العزلُ غرضين معاً: منع تسرّب حركة الهجوم إلى الشبكات الحقيقية، وإتاحة الوسم التلقائي بالاعتماد على ثبات العنوان المصدري.

مخطّط المكونات

من البنية العامة إلى البنية البرمجية، يعرض الشكل 4 الوحدات البرمجية وما تنتجه وتستهلكه من ملفّات.

Component / Module Diagram



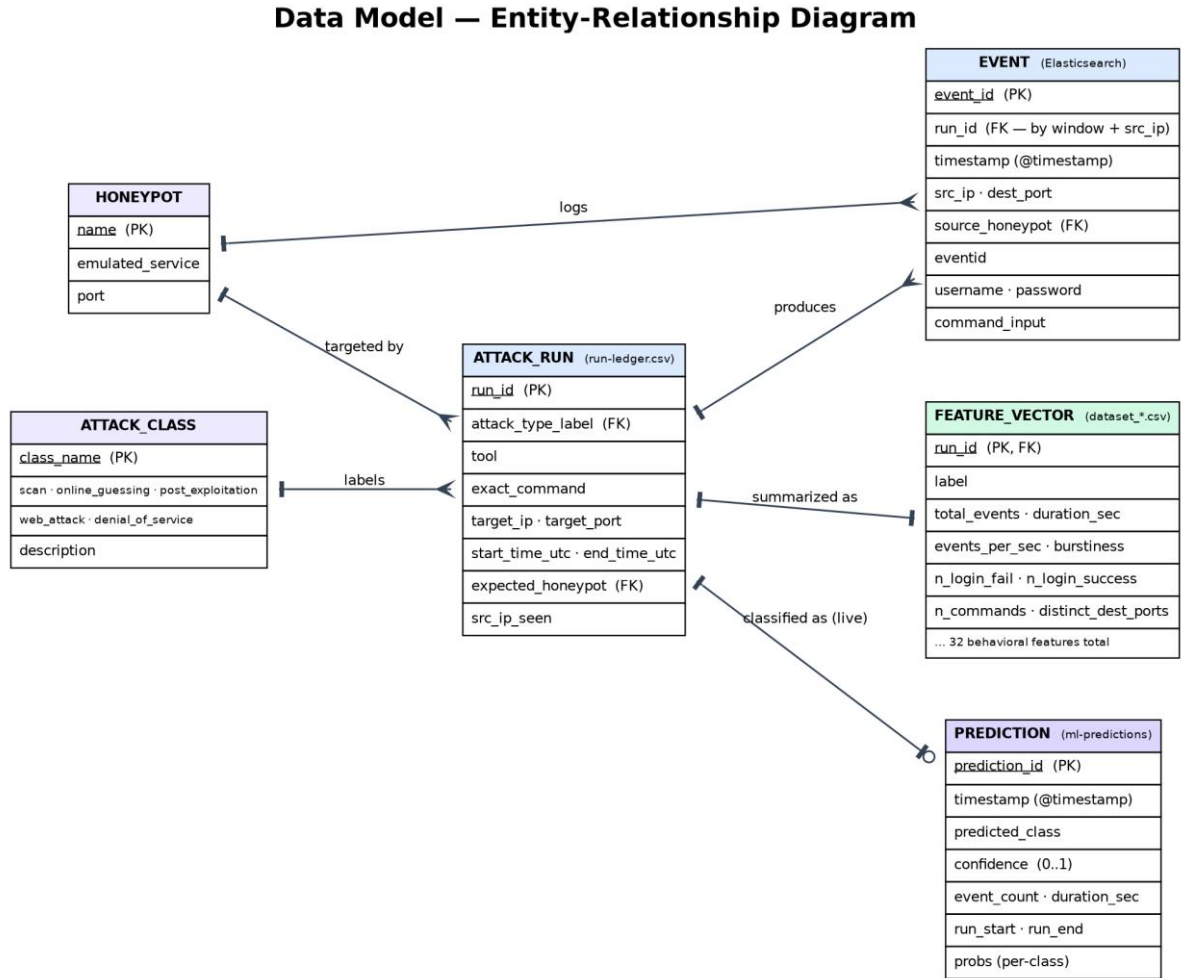
الشكل 4: مخطط المكونات والوحدات البرمجية.

يوضح المخطط تدفق العمل بين الوحدات: نصوص الحملات تكتب في سجل التشغيل، ونص التصدير يحول أحداث Elasticsearch إلى ملفات JSON لكل جولة، ومستخرج السمات يبني مجموعات البيانات، والدفاتر تدرب النماذج وتحفظها، وأخيراً يحمل `live_runs.py` النموذج ويكتب التنبؤات. وأبرز علاقة في المخطط أنّ `live_runs.py` يستورد `extract_features.py` نفسه، فتُحسب سمات التشغيل الحي بالرمز نفسه

المستعمل في التدريب (feature parity). وبهذا يكون الخطّ معيارياً قابلاً لإعادة التشغيل، ويضمن إعادة استعمال المستخرج تطابق التدريب والتشغيل.

نموذج البيانات

يوضّح الشكل 5 كيف تُنمذج بيانات المشروع وعلاقاتها في مخطّط الكيانات والعلاقات (ERD).



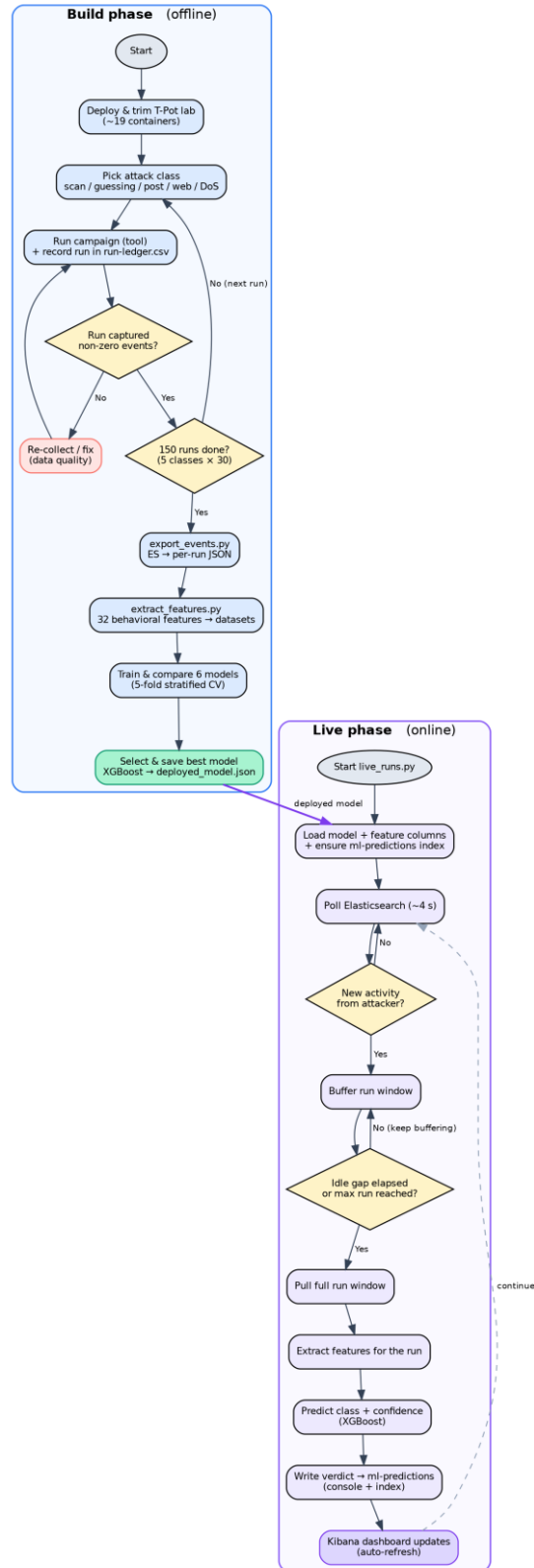
الشكل 5: نموذج البيانات (مخطّط الكيانات والعلاقات).

تُنمذج البيانات في ستّة كيانات: فئة الهجوم (ATTACK_CLASS) تسمّ جولاتٍ كثيرة، والجولة الواحدة (ATTACK_RUN) تُنتج أحداثاً كثيرة (EVENT) وتُخصّص في متّجه سماتٍ واحد (FEATURE_VECTOR)، وتُصنّف الجولة الحيّة إلى تنبؤٍ واحد أو لا شيء (PREDICTION)، ويسجّل كلّ honeypot أحداثاً كثيرة. ونشير إلى أنّ هذا نموذج مفاهيمي للبيانات فوق سجلّ CSV وElasticsearch وفهرس التنبؤات، لا مخطّط قاعدة بياناتٍ علائقية. ويظهر المخطّط كيف يتحوّل الهجوم الخام إلى صفّ تدريبٍ واحد موسوم، وإلى تنبؤٍ واحد عند التشغيل الحي.

مخطّط التدفقّ الشامل

يعرض الشكل 6 الخطّ الكامل للعمل مقسماً إلى طورٍ للبناء (offline) وطورٍ للتشغيل الحي (online).

End-to-End Pipeline — Flowchart



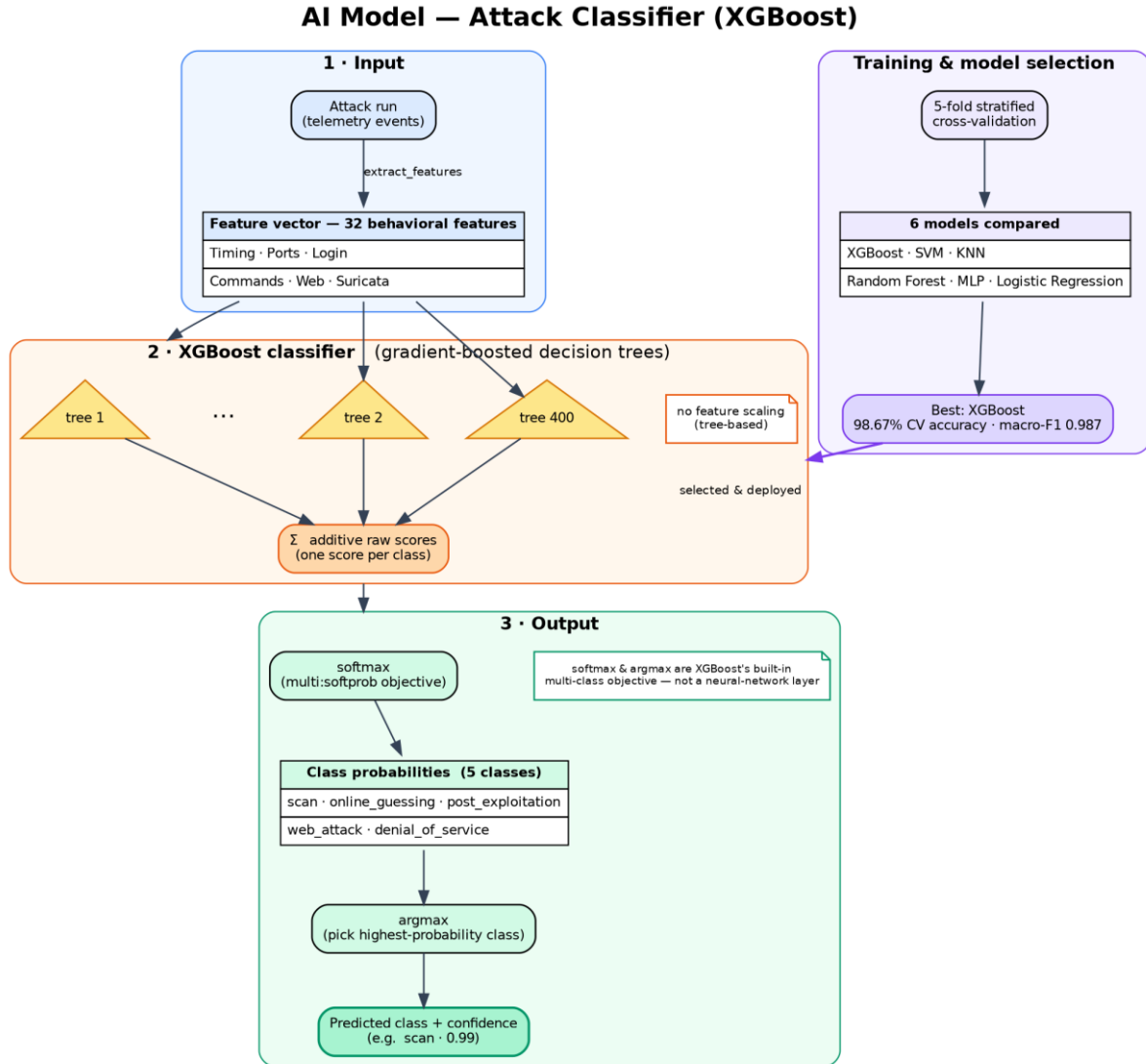
الشكل 6: مخطط تدفق الخط الكامل (البناء والتشغيل الحي).

في طور البناء تظهر حلقتان مهمتان: حلقة إعادة الجمع لمعالجة جودة البيانات، وحلقة جمع الجولات المئة والخمسين، ثم يأتي التصدير فاستخراج السمات فتدريب النماذج ومقارنتها بحفظ النموذج. وفي طور التشغيل الحي تظهر حلقة كشف

الخمول التي تحدّد نهاية الجولة. ومواضع القرار في المخطّط — التقاط أحداثٍ غير صفرية، واكتمال المئة والخمسين، ومرور فجوة الخمول — هي بالضبط المواضع التي يضمن فيها الخطّ جودة البيانات وصحة حدود الجولات.

مخطّط نموذج الذكاء الاصطناعي

يوضّح الشكل 7 بنية المصنّف وكيفية إنتاجه للتنبؤ، وهو قلب النظام.



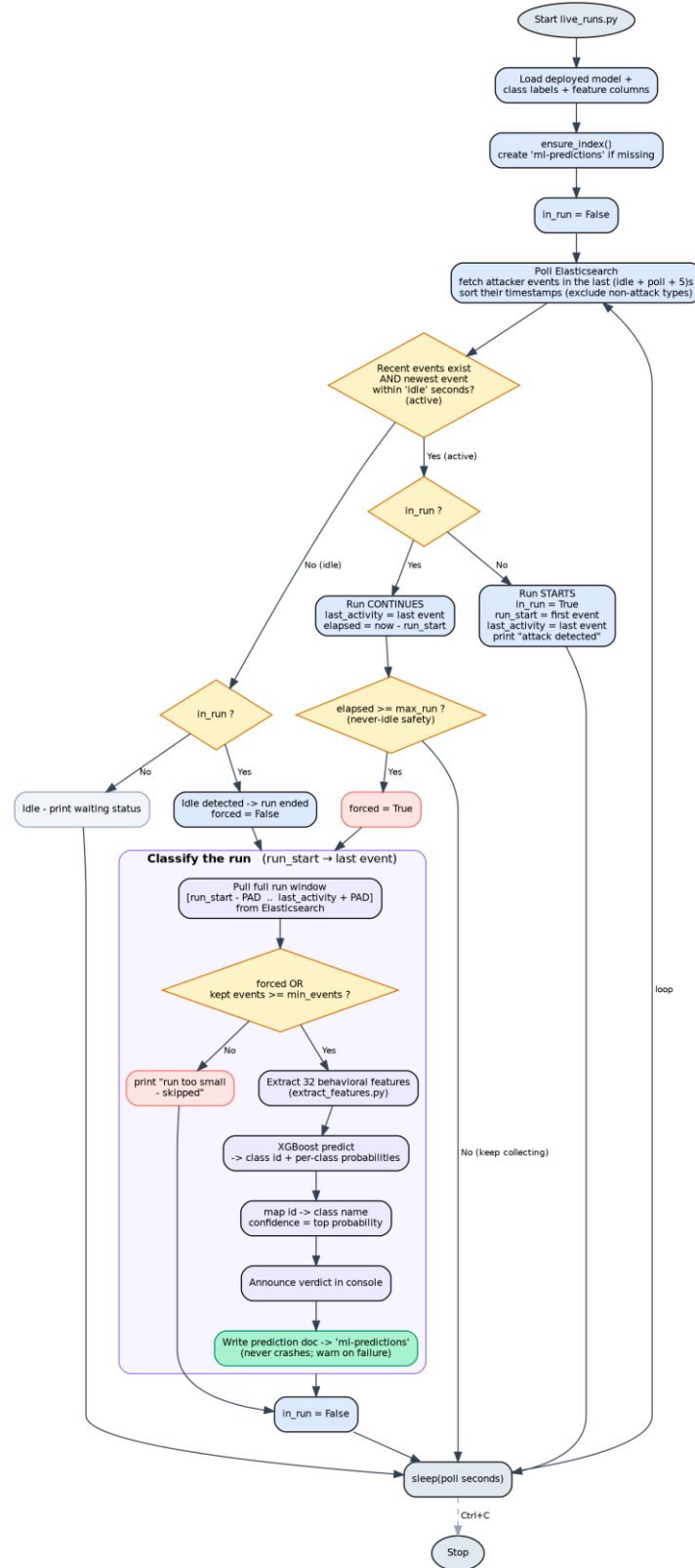
الشكل 7: مخطّط نموذج الذكاء الاصطناعي (مصنّف XGBoost).

يأخذ النموذج متّجه السمات (٣٢ سمة سلوكية) ويمزّره على مجموعة من الأشجار المعزّزة بالتدرّج (gradient-boosted trees) عددها نحو أربعمئة شجرة، فتُجمّع درجاتها الخام لكلّ فئة، ثم تمرّ على دالة softmax (وهي هدف multi:softmax في XGBoost)، فتنتقى الفئة الأعلى احتمالاً بدالة argmax، لتخرج الفئة المنتبأ بها مع درجة الثقة. ويبين المخطّط في جانبه أنّ النموذج اختير عبر التحقق المتقاطع من بين ستّة نماذج. وننبّه إلى أنّ softmax و argmax هنا جزء من هدف XGBoost المدمج للتصنيف متعدّد الفئات، لا طبقة في شبكة عصبية. والأهم أنّ النموذج يقرأ سلوك الهجوم، وأنّ السمات الاثنتين والثلاثين نفسها تُغذي التدريب والتشغيل الحي معاً.

مخطّط تدفق التصنيف الحي

يعرض الشكل 8 منطق المصنّف الحي في كشف جولات الهجوم الكاملة وتصنيفها.

Live Classification — Idle-Bounded Run Detection (live_runs.py)

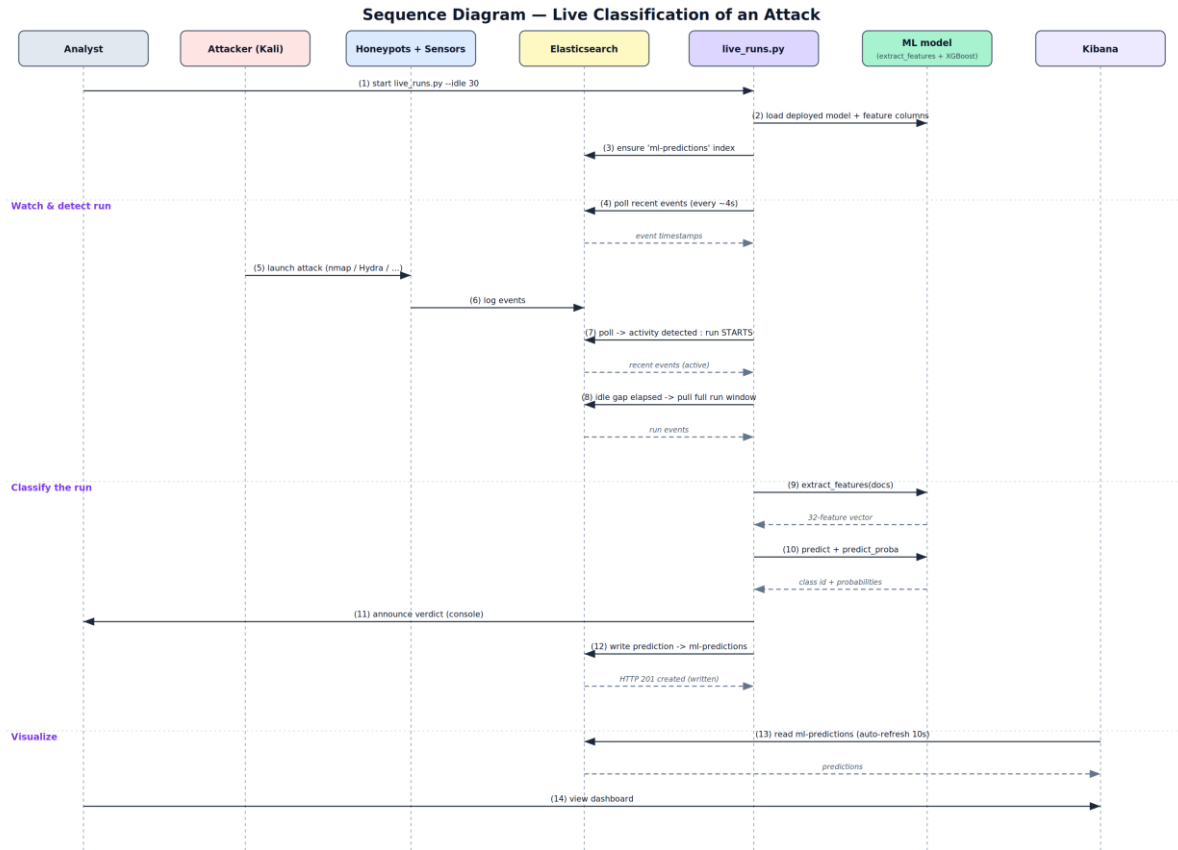


الشكل 8: مخطط تدفق التصنيف الحي (كشف الجولات المحدود بالخمول).

يقوم المصنّف على آلة حالات (state machine) محورها العلم `in_run`: تبدأ الجولة عند ظهور نشاط، وتستمرّ ما دام النشاط قائماً، وتُصنّف حين تمرّ فجوة الخمول. ويظهر في المخطّط الحدّ الأقصى لطول الجولة (`max_run`) الذي يفرض التصنيف للفيضانات المتواصلة، وبوابة الحدّ الأدنى للأحداث (`min_events`)، وسحب نافذة الجولة الكاملة بهامش زمني، وكتابة التنبؤ التي لا تُسقط المصنّف أبداً. وميزة تحديد الجولة بفترات الخمول أنّها تطابق الطريقة التي تُربّ بها النموذج (على جولاتٍ كاملة) دون الحاجة إلى نافذة منزلة ثابتة.

مخطّط التتابع

يتتبّع الشكل 9 تسلسل التفاعل زمنياً أثناء اكتشاف هجوم وتصنيفه.



الشكل 9: مخطّط التتابع للتصنيف الحي.

ينقسم التفاعل إلى أربعة أطوار: التهيئة، ثم المراقبة وكشف الجولة، ثم التصنيف، ثم العرض. ويوضّح المخطّط من يستدعي من: يتبادل `live_runs.py` الطلبات مع Elasticsearch (الاستطلاع الدوري، ثم سحب نافذة الجولة)، ثم مع نموذج التعلم الآلي (استخراج السمات والتنبؤ)، ثم يكتب النتيجة، فتقرأها Kibana وتعرضها. وتبرز في المخطّط أزمنة العمل: الاستطلاع الدوري، وانتظار فجوة الخمول، ثم تصنيف واحد وكتابة واحدة لكلّ جولة.

التنفيذ والاختبارات

مقدمة

يعرض هذا الفصل الجانب الهندسي من المشروع: المعمارية التي بُني عليها كل جزء من النظام المنفذ، وتفصيل أجزائه وكيفية تكاملها بعضها مع بعض، ثم الشيفرة الوصفية (pseudocode) لأهم خوارزمياته، وأخيراً خطة الاختبارات التي جرى بها التحقق من صحته وأمنه واستقراره تحت الحمل. والغاية أن يتضح كيف بُني النظام وكيف تأكدنا من عمله، لا ما يفعله فحسب.

١. معمارية النظام المتبعة

١.١ النمط المعماري العام

يقوم النظام على نمطين متكاملين: خط معالجة بيانات (data pipeline) يعمل دفعةً (batch/offline)، وخدمة تصنيف حية (online service). يمر خط المعالجة بمراحل متتابعة ومنفصلة — توليد الهجوم، ثم الالتقاط والتخزين، ثم التصدير، ثم استخراج السمات، ثم التدريب — يتدفق فيها الاتجاه في مسار واحد. والمهم في هذا النمط أن المراحل مفصولة عبر نظام الملفات وElasticsearch لا عبر استدعاءات مباشرة بينها؛ فكل مرحلة نص برمجي (script) مستقل يقرأ مخرجات المرحلة السابقة من القرص ويكتب مخرجاته إليه. ويمنح هذا الفصل ثلاث خصائص: أن كل مرحلة قابلة لإعادة التشغيل على حدة، وأنها عديمة الأثر (idempotent) فلا تكرر عملاً أنجز، وأنها قابلة للاستئناف بعد أي انقطاع.

١.٢ معمارية الالتقاط والتخزين

تتبع مرحلة الالتقاط نمط «مُنتج – مُجمّع – مخزن». الـ honeypots والمستشعرات (Suricata وp0f وfatt) مُنتجات مستقلة للأحداث، تُرسلها إلى Logstash الذي يجمعها ويشحنها إلى Elasticsearch، فيبقى Elasticsearch العمود الفقري للتخزين والبحث الذي تُبنى عليه كل المراحل اللاحقة. وميزة هذا الفصل أن إضافة honeypot جديد أو مستشعر لا تمس بقية النظام، وأن البيانات تبقى متاحة للاستعلام مستقلة عن الأداة التي أنتجتها.

١.٣ معمارية توليد البيانات والوسم

اعتمدنا نمط مُشغل حملة لكل فئة (campaign runner): نص برمجي لكل نوع هجوم يولد قائمة متنوعة من الأوامر، ويغلف أداة الهجوم بمؤقت زمني (timeout)، ويسجل كل جولة في سجل التشغيل مع نافذتها الزمنية الدقيقة. والأصل المعماري اللافت هنا أن الوسْم يقوم على السياق لا على تعديل الأداة: تُنسب الأحداث إلى الجولة بمطابقة نافذتها الزمنية (UTC) والعنوان المصدري 1.32.168.192، وتضمن فترات التهدئة (cooldown) عدم تداخل النوافذ، فيصبح الوسْم قاطعاً دون الحاجة إلى أي تعديل على أدوات الهجوم نفسها.

١.٤ معمارية المعالجة (التصدير واستخراج السمات)

تتألف المعالجة من مرحلتين تفصل بينهما ملفتان وسيطة: التصدير الذي يسحب أحداث كل جولة من Elasticsearch إلى ملف JSON مستقل، ثم استخراج السمات الذي يحول ملفات JSON إلى صفوف رقمية في مجموعتي البيانات. وهذا التدرج المبني على الملفات (per-run JSON) بوصفه مرحلة وسيطة هو ما يجعل المعالجة عديمة الأثر وقابلة

للاستئناف: يتخطى التصدير أيّ جولة صُدّرت من قبل، ويعيد المستخرج بناء المجموعتين كاملتين في كلّ تشغيل انطلاقاً من الملفات المخزّنة.

١.٥ معمارية التدريب والنموذج

يفصل التصميم بين التدريب (يجري في دفاتر Jupyter، مرحلة offline) والتشغيل (يجري في المصنّف الحي). تُنتج مرحلة التدريب أثراً واحداً محدداً هو النموذج المُسلّس مع بياناته الوصفية، ويُحفظ بصيغة قابلة للنقل (صيغة XGBoost الأصلية) مستقلة عن إصدارات المكتبات والأجهزة. وبهذا الفصل يمكن إعادة التدريب أو استبدال النموذج دون المساس بالخدمة الحيّة، والعكس.

١.٦ معمارية التصنيف الحي والعرض

المصنّف الحي هو خدمة مدفوعة بالأحداث تقوم على آلة حالات مع استطلاع دوري (polling state machine): يراقب الأحداث الواردة، ويجمع الجولة، ويصنّفها عند اكتمالها. وتحكمه ثلاثة مبادئ معمارية: تماثل السمات (feature parity) — إذ يستورد المصنّف الحي مستخرج السمات نفسه المستعمل في التدريب، فتُحسب السمات بالشفرة عينها ويُستبعد أيّ انحراف بين التدريب والتشغيل؛ والتدهور اللطيف (graceful degradation) — إذ لا تُسقط كتابة التنبؤ المصنّف أبداً، فالتصنيف هو المُنتج والعرض إضافةً مكّلة؛ وفصل القراءة عن الكتابة — إذ يكتب Python التنبؤات في فهرس ml-predictions، وتكتفي Kibana بقراءة ذلك الفهرس وعرضه، فلا تشغّل Kibana أيّ نموذج.

١.٧ خلاصة القرارات المعمارية

تتلخّص القرارات التي حكمت التصميم في: الفصل بين المراحل عبر الملفات وElasticsearch؛ وجعل كلّ مرحلةٍ عديمة الأثر وقابلة للاستئناف؛ وتماثل السمات بين التدريب والتشغيل؛ وحفظ النموذج بصيغة قابلة للنقل؛ والتدهور اللطيف في المسار الحي؛ وفصل القراءة عن الكتابة في العرض؛ وحدود سمات آمنة من التسريب. وهذه القرارات مجتمعةً هي ما جعل النظام قابلاً للنشر والتوظيف.

٢. تفصيل أجزاء النظام وتكاملها

٢.١ أجزاء النظام

يتكوّن النظام من الأجزاء التالية، ولكلٍّ منها مسؤولية واحدة واضحة:

١. **مُشغلات الحملات** (run_postexploit، run_online_guessing، run_campaign): توليد الهجمات وتسجيلها.
٢. **نصوص التحقق** (verify_*.py): التأكد من سلامة الالتقاط لكلّ فئة.
٣. **مُصدّر الأحداث** (export_events.py): سحب أحداث كلّ جولة من Elasticsearch إلى JSON.
٤. **مُستخرج السمات** (extract_features.py): تحويل الأحداث إلى صفوف سمات.
٥. **دفاتر التدريب** (03-01): تدريب النماذج ومقارنتها واختيار الأفضل وحفظه.
٦. **المصنّف الحي** (live_runs.py): كشف الجولات الحيّة وتصنيفها وكتابة التنبؤات.

7. منظومة ELK ولوحة Kibana: التخزين والعرض.

٢.٢ عقود التكامل بين الأجزاء

يجري التكامل عبر عقود بيانات (data contracts) واضحة بين المراحل لا عبر ربط مباشر، وهو ما يجعل الأجزاء قابلة للتبديل والاختبار على حدة. يلخّص الجدول التالي هذه العقود:

الجدول 3: عقود البيانات (data contracts) بين أجزاء النظام.

التكامل (من ← إلى)	آلية الربط والعقد (interface / contract)
مُشغّل الحملة ← السجلّ	ملفّ run-ledger.csv بأعمدته الأحد عشر؛ والنافذة الزمنية مع src_ip هي مفتاح ربط الجولة بأحداثها
السجلّ + Elasticsearch ← التصدير	استعلام ES: نافذة الجولة ±٣ ثوانٍ + مرشّح src_ip + استبعاد بورنات الإدارة، عبر Scroll API
التصدير ← مُستخرج السمات	ملفّ JSON لكلّ جولة في مجلّد es-events؛ والسجلّ يوفّر الوسم (run_id → label) فقط لا غير
المُستخرج ← الدفاتر	ملفًا dataset_behavioral.csv و dataset_full.csv بترتيب أعمدة ثابت
الدفاتر ← المصنّف الحي	+ deployed_model.json + feature_columns.json class_labels.json (النموذج، وترتيب السمات، وخريطة الفئات)
المصنّف الحي ← Kibana	وثيقة تنبؤ في فهرس ml-predictions بتخطيط محدّد (date / keyword / float)
التدريب ↔ التشغيل	استيراد extract_features.py نفسه في المصنّف الحي (تماثل السمات على مستوى الشيفرة)

ونشير إلى أنّ أهمّ عقدين هما feature_columns.json الذي يضمن تطابق ترتيب السمات بين التدريب والتشغيل، وتماثل السمات على مستوى الرمز الذي يمنع أيّ اختلاف في طريقة حساب السمات؛ فبهذين يُضْمَن أنّ ما يراه النموذج حيّاً مطابق لما دُرّب عليه.

٣. الشيفرة الوصفية (Pseudocode)

نعرض فيما يلي الشيفرة الوصفية لأهمّ خوارزميات النظام، مكتوبة لتطابق منطق النصوص البرمجية الفعلية.

٣.١ مُشغّل الحملة

يبنى قائمة الأوامر المتنوّعة، ويستأنف من حيث توقّف، ويسجّل كلّ جولة بنافذتها:

```

FUNCTION run_campaign(label, prefix):
    commands = build_attack_commands()           # techniques x port-scopes +
    heavy variants                                # resume from where we
    done      = count_ledger_rows(prefix)
    stopped
    FOR command IN commands[done:]:

```

```

run_id = prefix + next_index
start = utc_now()
output = run_tool_with_timeout(command, timeout = 900s)
end = utc_now()
save_transcript(run_id, output)
append_to_ledger(run_id, label, tool, command,
                  target, port_scope, start, end,
                  expected_honeypot, src_ip = "192.168.32.1")
sleep(COOLDOWN) # guarantees non-overlapping
windows

```

٣.٢ الوسم التلقائي بالنافذة والمصدر

الوسم قاعدة منطقية لا خطوة يدوية:

```

event BELONGS TO run IFF
    run.start <= event.timestamp <= run.end
    AND event.src_ip == "192.168.32.1"
# cooldowns ensure windows never overlap -> labeling is unambiguous

```

٣.٣ تصدير الأحداث

يسحب أحداث كل جولة عبر Scroll API، ويتخطى ما صُدِّر سابقاً:

```

FUNCTION export_events():
    FOR run IN ledger:
        IF json_file(run) EXISTS with > 0 events: SKIP # idempotent
        query = { src_ip == "192.168.32.1"
                  AND @timestamp IN [run.start - 3s, run.end + 3s]
                  AND dest_port NOT IN management_ports }
        docs = elasticsearch_scroll("logstash-*", query, page = 2000)
        write_json(run, docs)
        append_manifest(run, count = len(docs))

```

٣.٤ استخراج السمات

يحوّل أحداث الجولة إلى متّجه سمات، مستبعداً التلوث، معتمداً على التلمتزية وحدها:

```

FUNCTION extract_features():
    labels = read_run_labels(ledger) # ledger gives ONLY run_id ->
    label
    FOR run_id IN labels:
        docs = drop(load_json(run_id) WHERE type == "NGINX") # remove
        contamination
        f = volume_features(docs) # total_events
        f += timing_features(docs) # duration, events/sec,
        burstiness, ...
        f += port_features(docs) # distinct dest/src ports,
        concentration
        f += auth_features(docs) # login fail/success,
        n_auth_other (nonempty pw)
        f += command_features(docs) # commands, sessions,
        distinct commands
        f += web_features(docs) # http events, :80 / :8080,
        distinct paths
        f += suricata_features(docs) # alerts, flows, severities,
        signatures

```

```

        behavioral_row = f # 32 features
(after dropping 2 constant)
        full_row = f + per_honeypot_counts(docs) # + 9 counts =
41 features
        write_csv(dataset_behavioral, behavioral_rows)
        write_csv(dataset_full, full_rows)

```

٣.٥ تدريب النماذج واختيار الأفضل

يقارن ستة نماذج بالتحقق المتقاطع، ثم يعيد تدريب الأفضل ويحفظه:

```

FUNCTION train_and_select(dataset):
    X, y = load(dataset); y = encode_labels(y, FIXED_ORDER)
    FOR model IN {XGBoost, SVM, KNN, RandomForest, MLP, LogisticRegression}:
        pipeline = scale_if_needed(model) # StandardScaler for
distance/gradient models
        scores = stratified_k_fold_cv(pipeline, X, y, k = 5) # accuracy,
macro-F1
        errors = misclassified_runs(cross_val_predict(pipeline, X, y))
        record(model, scores, errors)
    best = argmax_accuracy(records) # -> XGBoost, 0.9867
    best.fit(X, y) # refit on all 150 rows
    save(best -> deployed_model.json, feature_columns.json,
        class_labels.json, deployed_model_meta.json)

```

٣.٦ التصنيف الحي (كشف الجولات المحدود بالخمول)

آلة حالات تبدأ الجولة عند النشاط وتصنفها عند الخمول، مع حد أقصى للطول:

```

FUNCTION live_runs(idle, poll, min_events, max_run):
    model, cols, labels = load_deployed_model()
    ensure_index("ml-predictions")
    in_run = False
    LOOP forever:
        times = poll_recent_timestamps(window = idle + poll + 5s) #
attacker events only
        active = times AND (now - last(times)) < idle
        IF active AND NOT in_run: # RUN STARTS
            in_run = True; run_start = first(times); last_activity =
last(times)
        ELIF active AND in_run: # RUN CONTINUES
            last_activity = last(times)
            IF (now - run_start) >= max_run: # never-idle safety
                classify_run(run_start, last_activity, force = True); in_run
= False
        ELIF in_run AND NOT active: # IDLE -> run ended
            classify_run(run_start, last_activity, force = False); in_run =
False
        sleep(poll)

FUNCTION classify_run(run_start, last_activity, force):
    docs = pull_window([run_start - PAD, last_activity + PAD])
    IF NOT force AND count(docs) < min_events: RETURN # ignore
trivial noise
    features = extract_features(docs) # SAME extractor as training
(parity)
    class_id, probs = model.predict(features) # XGBoost multi:softprob
    verdict = (labels[class_id], max(probs))
    announce(verdict)

```

```
write_to_index("ml-predictions", verdict, probs, meta) # never  
raises -> warn only
```

١. بيئة العمل والبنية التحتية

١.١ الأجهزة والشبكة

بُني المختبر بالكامل على جهاز واحد، بحيث تعمل عليه آلتان افتراضيتان معزولتان: الأولى تستضيف منظومة الـ honeypot، والثانية هي آلة المهاجم.

فيما يخص عناوين الشبكة:

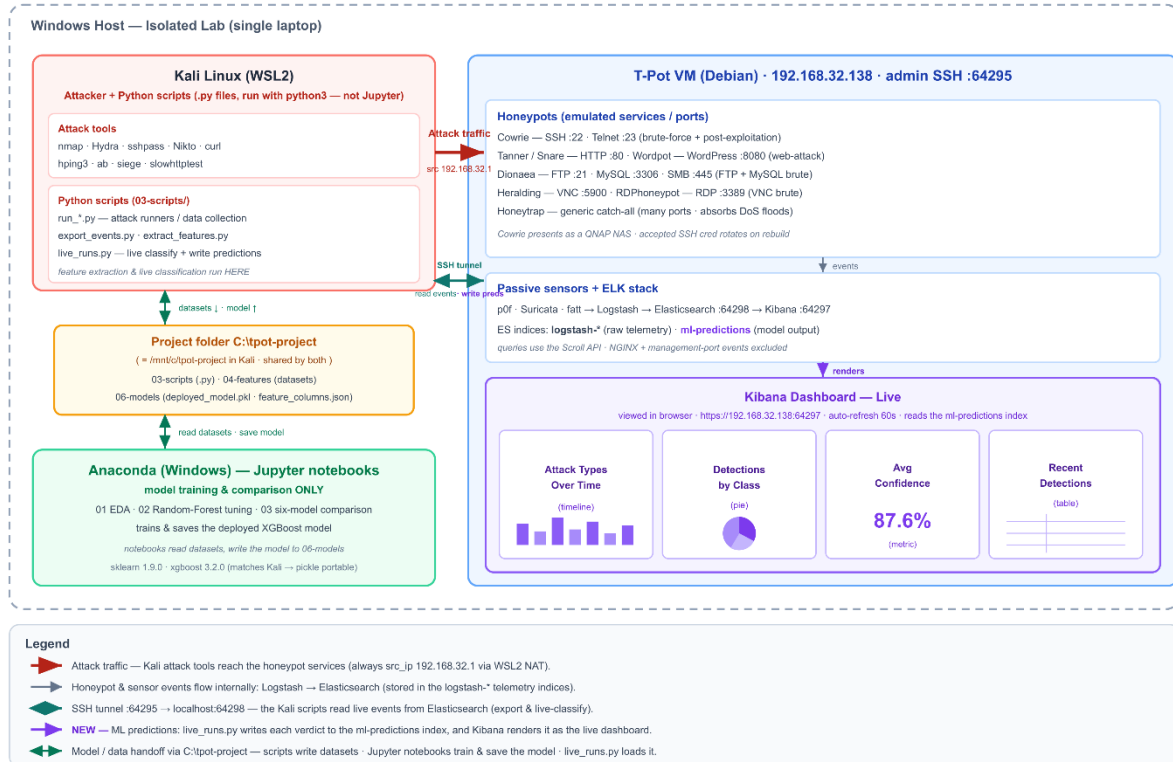
- آلة الـ **honeypot**: تعمل على نظام Debian بعنوان 192.168.32.138، وعليها تم تنصيب منظومة T-Pot بأكملها.

- آلة المهاجم: نظام Kali يعمل ضمن بيئة WSL2 على Windows، ومنه تُطلق جميع أدوات الهجوم.

من الأمور المهمة التي تنبّهنا إليها مبكراً واعتمدنا عليها في كل ما يلي: أنّ حركة المرور الصادرة من Kali لا تصل إلى الـ honeypot بعنوانها الظاهر داخل WSL2، بل تصل دائماً بالعنوان المصدري 192.168.32.1. والسبب أنّ WSL2 يشغّل شبكة افتراضية داخلية خاصّة به داخل Windows، فتُترجم الاتصالات الخارجة لتظهر وكأنّها صادرة عن Windows نفسه قبل أن تبلغ آلة الـ honeypot. ولهذا اعتمدنا العنوان 192.168.32.1 مرشحاً (filter) ثابتاً في كلّ استعلام لاحق على البيانات — فهو النقطة التي تُنسب إليها كلّ حركة المهاجم.

Lab Topology — T-Pot Honeypot + ML Attack Classification

Isolated single-laptop lab: attack → collect telemetry → train model (Jupyter) → live-classify (script) → write predictions → Kibana dashboard



الشكل 10: مخطط بيئة الاختبار المعزولة بوضوح آلة الـ honeypot وآلة Kali topology والشبكة.

١.٢ نشر T-Pot وتقليص الحاويات

يأتي T-Pot في هيئة منظومة قائمة على الحاويات (containers) تُدار عبر ملف `docker-compose.yml`، وتضمّ في تنصيبها القياسي (STANDARD / HIVE) عدداً كبيراً من الـ honeypots المتخصصة إلى جانب مكّنس التحليل. وقد أظهر التنصيب الأول الكامل لدينا ٣٩ حاوية تعمل في آن واحد.

هذا العدد الضخم يستهلك موارد كبيرة، وأغلب حاوياته لا يخدم أهداف مشروعنا. لذلك قلّصنا المنظومة إلى ما يقارب ٢٠ حاوية، بأن أبقينا فقط ما يلزم لتغطية أنواع الهجوم المستهدفة إلى جانب حاويات التحليل والعرض، وعطّلنا الباقي. تمّ ذلك بالتعليق (commenting-out) على كتلة كلّ خدمة غير مطلوبة في ملف `docker-compose.yml`، مع مراعاة أن يبقى مقدار الإزاحة (indentation) لكلّ كتلة مطابقاً لبقية الملف؛ إذ إنّ أيّ اختلاف ولو بمسافة واحدة يُفشّل تحليل الملف بأكمله.

يوضّح الجدول التالي ما أبقيناه وسبب إيقافه:

الجدول 4: حاويات T-Pot التي أُقيمت بعد التقليص وسبب إيقاف كلٍّ منها.

الغرض	الحاويات
النواة الأساسية	tpotinit,nginx
التحليل والعرض	elasticsearch,logstash,kibana , suricata,p0f,fatt
أهداف المسح (scan)	honeytrap,dionaea
أهداف التخمين عبر الإنترنت (online guessing)	VNC heralding, (SSH/Telnet) cowrie وبروتوكولات البريد، rdphoneypt (RDP)
أهداف الدخول المزيف (fake login)	wordpress (صفحة دخول WordPress مزيفة)، ومجموعة tanner + snare (tanner_api +) لتطبيق (tanner_redis, tanner_phpox ويب مليء بالثغرات

أمّا ما عطّلناه فيشمل خريطة الهجمات الجغرافية (لأنّ المهاجم هو الجهاز نفسه، فلا معنى لعرض موقعه)، وأدوات إثراء المعلومات مثل spiderfoot وewsposter، إضافةً إلى نحو خمس عشرة حاوية لمحاكاة بروتوكولات نادرة أو أنظمة تحكّم صناعي (ICS/SCADA) لا تندرج ضمن فئاتنا. وطبقنا التغييرات بإعادة تشغيل المكّنس (docker compose down ثمّ docker compose up -d)، وتحقّقنا من سلامة الملف قبل ذلك بالأمر docker compose config -q.

جدير بالذكر أنّ مجموعة tanner/snare تُعامل كوحدة واحدة لا تتجزّأ، إذ يعمل snare كواجهة أمامية للويب بينما يتولّى tanner توليد الصفحات المصابة وتقييم الطلبات، ويعتمد tanner بدوره على حاوياته المساندة الثلاث؛ فإمّا أن تبقى المجموعة كاملةً أو تُعطّل كاملةً.


```

hsen@debian:~$ dps
NAME      STATUS      PORTS
cowrie    Up 20 minutes 0.0.0.0:22-23->22-23/tcp, [::]:22-23->22-23/tcp
dionaea   Up 20 minutes (healthy) 0.0.0.0:20-21->20-21/tcp, [::]:20-21->20-21/tcp, 0.0.0.0:42->42/tcp, [::]:42->42/tcp, 0.0.0.0:81->81/tcp, [::]:81->81/tcp, 0.0.0.0:135->135/tcp, [::]:135->135/tcp, 0.0.0.0:445->445/tcp, [::]:445->445/tcp, 0.0.0.0:1433->1433/tcp, [::]:1433->1433/tcp, 0.0.0.0:1723->1723/tcp, [::]:1723->1723/tcp, 0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp, 0.0.0.0:69->69/udp, [::]:69->69/udp
elasticsearch Up 20 minutes (healthy) 127.0.0.1:64298->9200/tcp
fatt      Up 20 minutes 0.0.0.0:110->110/tcp, [::]:110->110/tcp, 0.0.0.0:143->143/tcp, [::]:143->143/tcp, 0.0.0.0:465->465/tcp, [::]:465->465/tcp, 0.0.0.0:993->993/tcp, [::]:993->993/tcp, 0.0.0.0:995->995/tcp, [::]:995->995/tcp, 0.0.0.0:1080->1080/tcp, [::]:1080->1080/tcp, 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
heralding Up 20 minutes 0.0.0.0:110->110/tcp, [::]:110->110/tcp, 0.0.0.0:995->995/tcp, [::]:995->995/tcp, 0.0.0.0:1080->1080/tcp, [::]:1080->1080/tcp, 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
honeytrap Up 20 minutes
kibana    Up 18 minutes (healthy) 127.0.0.1:64296->5601/tcp
logstash  Up 18 minutes (healthy) 127.0.0.1:64305->64305/tcp
nglnx     Up 20 minutes 0.0.0.0:64294->64294/tcp, [::]:64294->64294/tcp, 0.0.0.0:64297->64297/tcp, [::]:64297->64297/tcp
p0f       Up 20 minutes
rdphoneypot Up 20 minutes 0.0.0.0:3389->3389/tcp, [::]:3389->3389/tcp
snare     Up 20 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp
suricata  Up 20 minutes
tanner    Up 20 minutes
tanner_api Up 20 minutes
tanner_phpx Up 20 minutes
tanner_redis Up 20 minutes
tpotinit  Up 20 minutes (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
wordpot   Up 20 minutes
hsen@debian:~$

```

الشكل 11: خرج الأمر `dps (docker ps)` يُظهر الحاويات العشرين العاملة بعد التقليل وحالتها.

١.٣ منظومة الرصد والتحليل

تقوم عملية جمع البيانات كلها على مكدس ELK المدمج في T-Pot، وهو مؤلف من ثلاثة مكونات متكاملة:

- **Elasticsearch** (النسخة 9.3.5): محرك التخزين والبحث الذي يستقبل كل حدث تسجله المنظومة ويخزنه ضمن فهرس باسم `*-logstash`.
- **Logstash**: المسؤول عن تجميع السجلات من مصادرها المختلفة والتعديل عليها وشحنها إلى **Elasticsearch**.
- **Kibana** (على البوابة 64297): واجهة الويب التي نعاين من خلالها البيانات ونبنى الاستعلامات والرسوم. إلى جانب الـ **honeypots** نفسها، تضيف ثلاث أدوات رصد على مستوى الشبكة تغطية أعمق: **Suricata** (نظام كشف اختراق (IDS) يسجل التدفقات (flows) والتنبهات (alerts) على مستوى الحزم)، و **P0f** (يبصم حركة المرور بصمة سلبية (passive fingerprinting) عبر خصائص حزم TCP)، و **Fatt** (يبصم اتصالات SSH). وهذا المزيج يعني أن الهجوم الواحد يُرصد من زوايا متعددة في آن واحد: من الـ **honeypot** الذي تعامل معه، ومن أدوات الشبكة التي راقبت الاتصال بمعزل عن أي منها.

ومن المفيد الإشارة إلى أن كل مستشعر يكتب حقولاً مختلفة في سجلاته؛ فمثلاً يسجل **Cowrie** حقولاً مثل `eventid` و `username` و `password` و `input` (الأمر المنفذ) و `session`، بينما يسجل **Suricata** حقولاً مثل `event_type` و `alert.signature`، ويسجل **P0f** معلومات البصمة ونظام التشغيل المُخْتَن. هذا التنوع في الحقول هو ما سنستثمره لاحقاً عند اشتقاق السمات السلوكية (behavioral features).

١.٤ الوصول إلى البيانات وتنقيتها

إن **Elasticsearch** مقيّد بالعنوان المحلي (`localhost:64298`) لأسباب أمنية، لذا كنّا نصل إليه من آلة **Kali** عبر نفق (SSH tunnel) **SSH** يمرّ ببوابة الإدارة 64295. واستعملنا صيغة نفق ذات إبقاء-على-قيد-الحياة (`keepalive`) حتى لا ينقطع الاتصال أثناء العمل، مع إغلاقه عمداً أثناء هجمات الفيضان (`floods`) كي لا يتسبّب ضغطها في إعادة تعيينه.

كذلك اعتمدنا مجموعة قواعد ثابتة لتنقية البيانات من كل ما ليس هجوماً، حرصاً على نظافة مجموعة البيانات:

1. استبعاد حركة **NGINX**: وهي في الواقع تصفحنا نحن للوحة Kibana (وتصدر أيضاً عن 192.168.32.1)، فاستبعدناها بوصفها تلوّثاً لا هجوماً.
 2. استبعاد بوابات الإدارة (النطاق 64294-64305): وهي حركة النفق والبنية التحتية.
 3. تجاهل وسم الموقع الجغرافي (**geoip**): إذ يلحق T-Pot بكلّ حدث عنوانه العام الذي اكتشفه لنفسه (يظهر منسوباً إلى «Syrian Telecom / دمشق»).
- أخيراً، نُظِّم المشروع على القرص في بنية مجلّدت واضحة تفصل بين سجلّ التشغيل، والبيانات الخام، والنصوص البرمجية (**scripts**)، والسّمات المستخرجة، والنماذج، ودفاتر Jupyter، بما يجعل خطّ العمل (**pipeline**) قابلاً لإعادة التشغيل.

١.٥ الخدمات التي تحاكيها الـ honeypots والمستشعرات

يشغل T-Pot نوعين مختلفين: **honeypots** تحاكي خدمةً فعلية (يتصل بها المهاجم ويتفاعل معها)، ومستشعرات تراقب الحركة فقط (لا تقدّم خدمة). والتمييز بينهما مهمّ، فالأولى هي «سطح الهجوم» الذي يدخل منه المهاجم، والثانية هي التي ترصده من الخارج. يلخّص الجدول التالي الـ honeypots التي تقدّم خدمة تفاعلية:

الجدول 5: الـ honeypots التي تقدّم خدمة تفاعلية والخدمات التي تحاكيها.

الـ honeypot	الخدمة التي يحاكيها	المنفذ	ملاحظة من المشروع
Cowrie	SSH و Telnet	22، 23	الأساسي: هدف التخمين عبر الإنترنت وما بعد الاختراق؛ يقبل زوجاً واحداً يتبدّل.
Snare/Tanner	خادم ويب HTTP	80	هدف هجوم الويب؛ يصنّف XSS فقط ويعدّ ما عداه حميداً.
Wordpot	موقع WordPress	8080	هدف ويب ثانٍ لفئة هجوم الويب.
Dionaea	FTP و MySQL و SMB	21، 3306، 445	سجلّ تخمين FTP عبر الإنترنت؛ يسجلّ المحاولات دون نتيجة نجاح/فشل (تغذي n_auth_other).
Heralding	VNC وبروتوكولات نصّية	5900	سجلّ تخمين VNC عبر الإنترنت
RDP honeypot	RDP (سطح المكتب البعيد)	3389	ردّه الملتبس بصمة كاشفة؛ لم يسجلّ محاولات دخول صالحة، فأعيد جمع تلك الجولات كـ SSH.
Honeytrap	النقاط عامّ لمنافذ كثيرة	منافذ متعدّدة	يفسر لماذا سجّلت فيضانات حجب الخدمة على منافذ متنوّعة (445، 5555، 1433، 53، ...).

أما **المستشعرات** فلا تقدّم خدمة بل تراقب: p0f يصمّم نظام تشغيل المصدر سلبياً من حزم TCP (وهو TCP فقط، ولذلك لم تنتج فيضانات ICMP شيئاً)، و Suricata نظام كشف اختراق الشبكة الذي يُنتج أحداث التدفق والتنبيهات (وتدفقاته تُسجلّ متأخرةً ٦٠-١١٠ ثانية، وهو ما شكّل توقيت جولات حجب الخدمة وفجوة الخمول)، و fatty يلتقط

بصمات مصافحات TLS/SSH. وبقية الحاويات (logstash, elasticsearch, nginx, tpotinit, kibana) بنية تحتية لا honeypots.

وتتطبق فئات الهجوم الخمس على هذه الخدمات انطباقاً نظيفاً: المسح (scan) يجتاحها جميعاً، والتخمين عبر الإنترنت (online guessing) وما بعد الاختراق (post-exploitation) على SSH في Cowrie (مع FTP و VNC) للتخمين عبر الإنترنت)، وهجوم الويب (web attack) على Tanner و Wordpot، وحجب الخدمة (DoS) على أي منفذ يُوجّه إليه (يلتقطه honeytrap غالباً).

٢. منهجية جمع البيانات

٢.١ تعريف العينة والوسم التلقائي

بُنيت مجموعة البيانات على مبدأ بسيط ومنضبط: العينة الواحدة هي جولة هجوم واحدة (attack run). ولم نضع الوسوم (labels) يدوياً، بل اعتمدنا وسماً تلقائياً يقوم على أمرين معاً: النافذة الزمنية (بتوقيت UTC) التي جرت فيها الجولة، والمرشح المصدري 192.168.32.1. أي أنّ كلّ حدث صادر عن هذا العنوان ضمن نافذة جولة ما يُنسب تلقائياً إلى وسم تلك الجولة.

ولضمان ألا تتداخل جولة مع أخرى، فصلنا بين كلّ جولتين بفترة تهدئة (cooldown)، وتحققنا لاحقاً من عدم وجود أي تداخل زمني بين النوافذ عبر الجولات كلّها.

سُجّلت تفاصيل كلّ جولة في ملفّ سجلّ التشغيل run-ledger.csv، وهو الملف الأساس لعملية الوسم. يحوي هذا السجلّ أحد عشر حقلاً لكلّ جولة، أهمّها: مُعرّف الجولة (run_id)، ووسم نوع الهجوم، والأداة المستخدمة، والأمر الفعليّ المنفذ، وعنوان الهدف وبوابته، ووقتنا البداية والنهاية بتوقيت UTC، والـ honeypot المتوقّع أن يلتقطها، والعنوان المصدريّ المُشاهد، وملاحظات.

٢.٢ النصوص البرمجية للحملات وتصدير البيانات

لأتمتة عملية الجمع، كُتب لكلّ فئة هجوم نصّ برمجيّ (script) خاصّ بها: run_campaign.py للمسح، و run_online_guessing.py للتخمين عبر الإنترنت، و run_postexploit.py لما بعد الاختراق، و run_webattack.py لهجمات الويب، و run_dos.py لحجب الخدمة. وقد صُمّمت هذه النصوص لتتشارك في خصائص مهمة تجعل عملية الجمع موثوقة وقابلة للاستئناف:

1. **قابلية الاستئناف:** يحسب كلّ نصّ برمجيّ عدد الجولات المُنجزة سابقاً (بعد صفوف بادئتها في السجلّ) ثمّ يُكمل من حيث توقّف، بما يسمح بإجراء الجمع على دفعات.
2. **التغليب بمؤقت (timeout):** تُغلّف كلّ أداة هجوم بمؤقت زمنيّ يضمن توقّفها بشكل نظيف عند تجاوز حدّ معيّن، فلا تعلق الجولة.
3. **حفظ الأثر:** يُحفظ لكلّ جولة نصّ خرج الأداة (transcript)، ويُضاف صفّ إلى السجلّ يتضمّن الأمر الفعليّ ونافذته الزمنية.

بعد تنفيذ الحملات، يتولّى النصّ البرمجيّ `export_events.py` سحب أحداث كلّ جولة من Elasticsearch وكتابتها في ملف JSON مستقلّ لكلّ جولة. ويطبّق هذا السكريبت المرشّح المصدريّ 192.168.32.1، ويستبعد حركة NGINX وبوابات الإدارة، ويوسّع نافذة كلّ جولة بمقدار ثلاث ثواني من كلّ طرف ($\pm 3s$ pad) احتياطاً. وأخيراً، تحقّقنا من سلامة كلّ جولة عبر نصوص برمجية للتحقّق (`verify_runs.py` ونصوص لكلّ فئة) تؤكّد أنّ كلّ جولة قد التقطت عدداً غير صفريّ من الأحداث، وأنّه لا وجود لصفوف مكرّرة أو نوافذ متداخلة.

٢.٣ حجم البيانات النهائي

بلغت مجموعة البيانات النهائية ١٥٠ جولة، موزّعة بالتساوي على خمس فئات، بواقع ٣٠ جولة لكلّ فئة: المسح (scan)، والتخمين عبر الإنترنت (online guessing)، وما بعد الاختراق (post-exploitation)، وهجوم الويب (web attack)، وحجب الخدمة (DoS). وقد صُدِّر من هذه الجولات ما يقارب 1.43 مليون حدث خام من Elasticsearch، وهي الأحداث التي بُنيت عليها لاحقاً مجموعتنا السمات. ويُلاحَظ التفاوت الكبير في عدد الأحداث بين جولة وأخرى — من بضعة أحداث في هجوم ما بعد الاستغلال إلى عشرات الآلاف في فيضانٍ عنيف — وهو تفاوتٌ سنراه لاحقاً يحمل بنفسه بصمةً مميزة لكلّ فئة.

٣. حملات الهجوم بأنواعها الخمسة

بعد تجهيز البيئة ومنهجية الجمع، نفّذنا خمس حملات هجوم، واحدة لكلّ فئة. صُمِّم كلّ نوع ليغطّي طيفاً من الأساليب والأدوات والأهداف لا سلوكاً واحداً مكرّراً، حتى يتعلّم النموذج لاحقاً نمط الفئة لا توقيعاً (signature) بعينه. نعرض فيما يلي كلّ فئة بأداتها وهدفها وما سجّل عنها.

٣.١ المسح (scan)

اعتمدنا في هذه الفئة أداة nmap بوصفها الأداة القياسية لاستكشاف الشبكات والمنافذ (ports). ولم نكرّر الأمر نفسه ثلاثين مرّة، بل ولّدنا ثلاثين جولة تتنوّع في محورين:

1. نوع المسح: المسح التزامني (-ss / SYN scan)، ومسح الاتصال الكامل (-st / connect scan)، وكشف إصدارات الخدمات (-sv / version detection)، والمسح العنيف (-a / aggressive)، ومسح UDP (-su)، مع قوالب توقيع (timing templates) مختلفة من T2- إلى T4-.
 2. نطاق المنافذ: من المنافذ الشائعة الخمسة، إلى أشهر 50 و 100 و 200 و 500 منفذاً، إلى النطاق 1-1000، وصولاً إلى مسح شامل لكلّ المنافذ (all-ports) وجولة UDP لأشهر 30 منفذاً.
- من طبيعة المسح أنّه نادراً ما يُكمل مصافحة (handshake) تطبيقية كاملة، لذا كان يُرصد غالباً على مستوى الشبكة عبر p0f و honeytrap و suricata، وأحياناً يلامس dionaea أو cowrie. وأظهرت نتائج nmap عدداً كبيراً من المنافذ المفتوحة على آلة T-Pot، يقابل كلّ منها خدمةٌ وهميّة يحاكيها أحد الـ honeypots (SSH على المنفذ 22، و HTTP على 80، وغيرها) — وهي في حقيقتها لافتات (banners) مزيفة لا خدمات فعلية.

```

(hsen@ Kali)-[mnt/c/tpot-project/03-scripts]
$ nmap 192.168.32.138
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-05 17:04 +0300
Nmap scan report for 192.168.32.138
Host is up (3.2s latency).
Not shown: 143 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
42/tcp    open  nameserver
80/tcp    open  http
81/tcp    open  hosts2-ns
110/tcp   open  pop3
135/tcp   open  msrpc
143/tcp   open  imap
445/tcp   open  microsoft-ds
465/tcp   open  smtps
993/tcp   open  imaps
995/tcp   open  pop3s
1024/tcp  open  kdm
1025/tcp  open  NFS-or-IIS
1026/tcp  open  LSA-or-nterm
1027/tcp  open  IIS
1028/tcp  open  unknown
1029/tcp  open  ms-lsa
1030/tcp  open  iad1
1031/tcp  open  iad2
1032/tcp  open  iad3
1033/tcp  open  netinfo
1034/tcp  open  zincite-a
1035/tcp  open  multidropper
1036/tcp  open  nsstp
1037/tcp  open  ams
1038/tcp  open  mtqp
1039/tcp  open  sbl
1040/tcp  open  netsaint
1041/tcp  open  danf-ak2
1042/tcp  open  afrog
1043/tcp  open  boinc
1044/tcp  open  dcutility

```

الشكل 12: نتيجة أمر nmap تُظهر المنافذ المفتوحة واللائحات (banners) الوهمية على آلة T-Pot.

٣.٢ التخمين عبر الإنترنت (online guessing)

استعملنا هنا أداة Hydra، وهي من أشهر أدوات هجمات التخمين عبر الإنترنت على معلومات الدخول (credentials). وورّعنا الثلاثين جولة على ثلاث خدمات مختلفة لتغطية أكثر من بروتوكول:

- SSH عبر Cowrie (خمس وعشرون جولة، المنفذ 22): جرّبنا أسماء مستخدمين متعدّدة (root،

admin، oracle، ubuntu) وقوائم كلمات مرور بأحجام مختلفة، مع تغيير عدد الخيوط المتوازية

(threads) بين 1 -t و 8 -t.

• FTP عبر Dionaeea (ثلاث جولات، المنفذ 21).

• VNC عبر Heraldng (جولتان، المنفذ 5900).

يُضبط Cowrie لقبول زوجاً واحداً صحيحاً من معلومات الدخول (يتبدّل عند إعادة بناء الحاوية)، ففي الجولات التي احتوت قائمتها على ذلك الزوج نجحت Hydra في الوصول إليه بعد سلسلة إخفاقات — منتجةً تسلسل «فشل ثم نجاح أو فشل فقط» واقعياً — بينما أنتجت الجولات التي استعملت مستخدماً غير مقبول أصلاً سلسلة إخفاقات صرفة. وكلا النمطين مفيد للنموذج. ويسجل Cowrie لكل محاولة حدثاً من نوع cowrie.login.failed أو cowrie.login.success يحمل اسم المستخدم وكلمة المرور المُجرَّبَين.

```
[hsen@Kali]~/mnt/c/tpot-project/03-scripts
$ hydra -t 4 -L wordlists/users.txt -P wordlists/pw_rec0l_ssh.txt ssh://192.168.32.138
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding
, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-05 17:09:03
[DATA] max 4 tasks per 1 server, overall 4 tasks, 600 login tries (l:10/p:60), ~150 tries per task
[DATA] attacking ssh://192.168.32.138:22/
[22][ssh] host: 192.168.32.138 login: root password: qwerty
[STATUS] 289.00 tries/min, 289 tries in 00:01h, 311 to do in 00:02h, 4 active
[STATUS] 261.50 tries/min, 523 tries in 00:02h, 77 to do in 00:01h, 4 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-05 17:11:25

[hsen@Kali]~/mnt/c/tpot-project/03-scripts
$
```

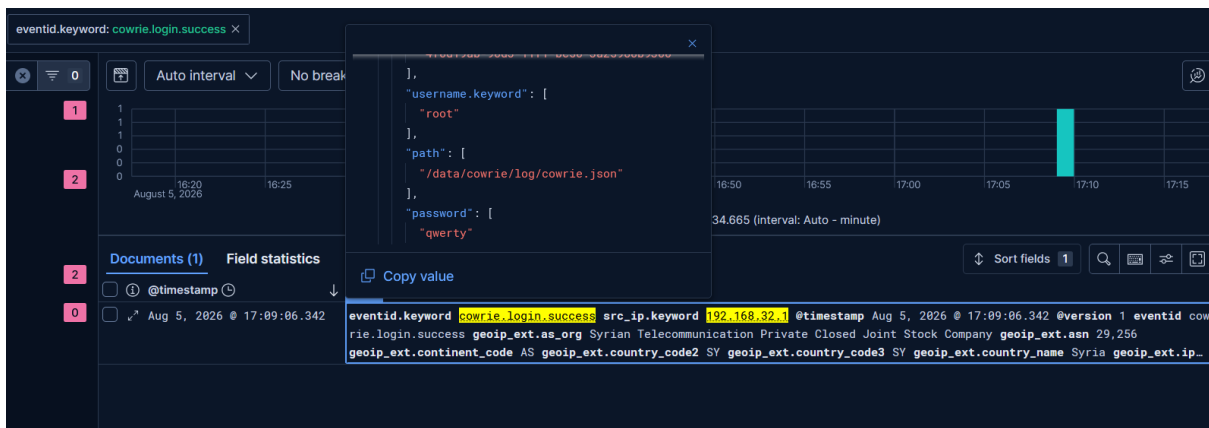
الشكل 13: خرج أداة Hydra في جولة SSH يُظهر سلسلة المحاولات ثم الزوج الناجح.



الشكل 14: كلمات المرور (password) التي جرّبها المهاجم حيث سجّلها Cowrie.



الشكل 15: أسماء المستخدمين (username) التي جربها المهاجم كما سجلها Cowrie.



الشكل 16: حدث cowrie.login.success يظهر الزوج الناجح root/qwerty.

نلاحظ كيف سجل cowrie كلمات السر وأسماء المستخدمين التي جربها المهاجم، كما سجل ال credentials الصحيحة وهي (username=root & password= qwerty)

٣.٣ ما بعد الاختراق (post-exploitation)

تحاكي هذه الفئة ما يفعله المهاجم بعد نجاحه في الدخول. ولتنفيذها، فتحنا جلسات SSH مؤتمتة (باستخدام sshpass) على الصدفة (shell) المزيفة التي يوفرها Cowrie على المنفذ 22، ثم نفّذنا في كلّ جولة سلسلة من الأوامر، مع تنويع طول الجلسة ومحتواها عبر الجولات الثلاثين. وغطّت الأوامر ثلاثة اتجاهات رئيسية:

1. الاستطلاع (reconnaissance): أوامر جمع معلومات النظام مثل a - و uname و id و whoami

و cat /etc/passwd و ps و ifconfig.

2. محاولات التمكين والبقاء (persistence): تنزيل حمولات (payloads) عبر curl/wget، أو الكتابة إلى ملفات، أو محاولات جدولة مهام.

3. طمس الأثر (cleanup): حذف سجلّ الأوامر ومحاولة إخفاء آثار الجلسة.

يحاكي Cowrie صدفة (shell) نظام Debian، ويسجلّ كلّ أمر يُدخّله المهاجم في الحقل input ضمن أحداث cowrie.command.input، وهو ما يمنحنا سجلاً كاملاً لسلوك المهاجم داخل الجلسة.

```
(hsen@ Kali)-[/mnt/c/tpot-project/03-scripts]
$ ssh root@192.168.32.138
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pg.html
OpenWrt 18.06.2
(root@192.168.32.138) Password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@OpenWrt:~# ls
root@OpenWrt:~# uname -a
Linux OpenWrt 4.14.95 #0 Mon Jan 28 08:54:02 2019 mips Linux
root@OpenWrt:~# whoami
root
root@OpenWrt:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/ash
daemon:x:1:1:daemon:/var:/bin/false
ftp:x:55:55:ftp:/home/ftp:/bin/false
nobody:x:65534:65534:nobody:/var:/bin/false
root@OpenWrt:~# ps
PID   TTY    TIME  COMMAND
1120   pts/0  0:00  ash
4577   pts/0  0:00  -bash
4579   pts/0  0:00  ps
root@OpenWrt:~# ifconfig
eth0    Link encap:Ethernet  HWaddr b8:ba:8c:a5:b1:e0
        inet addr:172.20.0.2  Bcast:172.20.0.255  Mask:255.255.255.0
        inet6 addr: fed8::160:62ff:fe2e:1701/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:149513 errors:0 dropped:0 overruns:0 frame:0
        TX packets:32134 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:130525449 (130.5 MB)  TX bytes:28053535 (28.1 MB)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING  MTU:65536  Metric:1
        RX packets:74054 errors:0 dropped:0 overruns:0 frame:0
        TX packets:74054 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:64649394 (64.6 MB)  TX bytes:64649394 (64.6 MB)
root@OpenWrt:~# exit
Connection to 192.168.32.138 closed.

(hsen@ Kali)-[/mnt/c/tpot-project/03-scripts]
$
```

الشكل 17: جلسة SSH لما بعد الاختراق داخل Cowrie تُظهر تسلسل أوامر الاستطلاع

Src IP - Top 10 - Dynamic		Cowrie Input - Top 10	
Source IP	Count	Command Line Input	Count
192.168.32.1	729	cat /etc/passwd	1
		exit	1
		ifconfig	1
		ls	1
		ps	1
		uname -a	1
		whoami	1

الشكل 18: لوحتا Kibana: العنوان المصدري، وأكثر أوامر . cowrie.command.input

نلاحظ كيف استطاع المهاجم الدخول الى ssh cowrie باستخدام ال credentials التي اكتشفها في hydra ونلاحظ كيف قام cowrie بتسجيل جميع الأوامر التي قام بتنفيذها المهاجم، وعنوانه المصدري.

٣.٤ هجوم الويب (web attack)

استهدفت هذه الفئة تطبيقات الويب في المنظومة، وتحديدًا مجموعة tanner/snare على المنفذ 80 و wordpot (المحاكي لموقع WordPress) على المنفذ 8080. واعتمدنا فيها أداتين متكاملتين:

- **Nikto (ثلاث جولات):** وهو ماسح ثغرات ويب (web vulnerability scanner) معروف. شغلناه على tanner وعلى wordpot بضبط فحص (-Tuning) وحدّ زمنيّ (-maxtime)، وغلفناه بمؤقت (timeout) لضمان توقّفه النظيف.
- **دفعات curl (سبع وعشرون جولة):** أرسلنا في كلّ جولة دفعةً (burst) من الطلبات المصمّمة يدويًا تحمل حمولات هجومية متنوّعة: حقن (SQL injection / SQLi) مثل `id=1' OR '1'='1` و `UNION SELECT`، والبرمجة النصّية العابرة للمواقع (XSS) مثل `<script>alert(1)</script>`، وإدراج الملفات المحليّ واجتياز المسارات (LFI / path) مثل `../../../../etc/passwd`، وحقن الأوامر (command injection) مثل `wp-login.php` و `wp-admin,xmlrpc.php` ضدّ wordpot. وتراوح حجم الدفعة بين ثلاثة طلبات وعشرين طلباً.

٣.٥ حجب الخدمة (denial of service / DoS)

هي الفئة الأوسع تنوعاً، إذ لا يكفي فيها توقيع (signature) واحد. صمّمنا الثلاثين جولة لتغطّي خمسة محاور من التعميم في آنٍ واحد: نوع الهجوم، والمنفذ المستهدف، والمعدل (rate)، والمدة، وحجم الحزمة (packet size) — كي يتعلّم النموذج «سلوك الإغراق» عموماً لا نمطاً بعينه. ووزّعناها على ثلاث عائلات من الأدوات:

1. hping3 (تسع عشرة جولة) — فيضانات على مستوى الشبكة: فيضانات (floods) من نوع SYN (-)

S على منافذ متعدّدة (80، 22، 445، 3306، 5555، 3389، 23، 1433، 8080) بمعدّلات وأعداد حزم مختلفة، وفيضانات بأعلام (flags) أخرى: ACK (-A) وFIN (-F) وXMAS (-F -P -U) وRST (-R)، وفيضانات UDP (--udp) على منافذ مثل 53 و123 و161 و500 و1883 و5060 بأحجام حمولة مختلفة.

2. ab و siege (ست جولات) — فيضانات HTTP: توليد كم كبير من طلبات HTTP المتزامنة

(concurrent) على المنفذين 80 و8080 لإغراق تطبيق الويب.

3. slowhttptest (خمس جولات) — هجمات الحجب البطيئة (slow / low-and-slow): إبطاء

الترويسات أو الأجسام (slow headers / slow body) لاستنزاف اتصالات الخادم وإبقائها مفتوحة أطول مدّة ممكنة.

وقد فرضت البيئة قيوداً عمليّة على هذه الفئة تعاملنا معها: تعدّر استخدام أداة nping، وعدم صلاحية حزم ICMP، وتعدّر تزوير العناوين (spoofing) تحت WSL2. كما ضبطنا المعدّلات بعناية كي لا تنهار الآلة الافتراضية نفسها تحت الضغط، وأضفنا داخل نافذة كلّ جولة فترة تصريف (drain) قصيرة، بعد أن لاحظنا أنّ Suricata يسجّل بعض أحداث التدفّق متأخّرة عن انتهاء الهجوم الفعلي.

```

(hsen@ Kali)-[/mnt/c/tpot-project/03-scripts]
$ python3 verify_DOS.py
=== per-run capture (src_ip=192.168.32.1) ===

[D001] SYN flood :80 high-rate (target 80) TOTAL=39811
sensors : P0f:21952, Suricata:17859
ports : 80:39791, 64295:20
suricata: alert:9561, flow:8295, ssh:3
distinct src_ports: 14906

[D002] SYN flood :22 med-rate (target 22) TOTAL=75967
sensors : P0f:49167, Suricata:26800
ports : 22:75967
suricata: alert:13448, flow:13352
distinct src_ports: 17188

[D003] SYN flood :445 med-rate (target 445) TOTAL=88678
sensors : P0f:53524, Suricata:35154
ports : 445:88666, 64295:10, 67:1, 137:1
suricata: alert:17584, flow:17569, ssh:1
distinct src_ports: 19457

[D004] SYN flood :3306 low-rate (target 3306) TOTAL=53515
sensors : P0f:34121, Suricata:19394
ports : 3306:53511, 64295:4
suricata: flow:9698, alert:9696
distinct src_ports: 9834

[D005] SYN flood :5555 high-rate (target 5555) TOTAL=85098
sensors : Suricata:42555, P0f:42543
ports : 5555:85098
suricata: alert:21383, flow:21172
distinct src_ports: 33330

[D006] SYN flood :3389 med-rate (target 3389) TOTAL=72717
sensors : P0f:42175, Suricata:30542
ports : 3389:72714, 64295:2, 67:1

```

الشكل 19: خرج نصّ التحقق verify_DOS.py يُظهر عدد الأحداث الملتقطة لكلّ جولة من جولات حجب الخدمة.

٤. معالجة جودة البيانات

الحصول على بيانات موثوقة وممثلة لكلّ فئة تطلّب معالجة عدد من المشكلات التي ظهرت أثناء الجمع. نعرض فيما يلي أبرزها، لأنّها تفسّر لماذا نظمنا إلى نظافة البيانات التي درّبت عليها النماذج.

٤.١ فساد عيّات ما بعد الاختراق (P016–P030)

عند فحص عيّات ما بعد الاختراق لاحظنا انقطاعاً حاداً يبدأ عند العيّنة P016 تحديداً: فبينما كانت العيّات السابقة سليمة، أظهرت P016 و P017 خمسة أحداث فقط بلا أيّ جلسة ولا محاولة دخول، وأظهرت ما بعدها رفضاً لمعلومات الدخول. تبين أنّ السبب هو إعادة تشغيل/بناء (restart/rebuild) لحاوية Cowrie في منتصف الجمع عند حدود P016، ممّا أوقف قبول معلومات الدخول مؤقتاً؛ فاصطدمت P016–P017 بحاوية قيد إعادة التشغيل، واصطدم ما بعدها بحاوية أعيد بناؤها فرفضت الدخول.

كان النص البرمجي `run_postexploit.py` سليماً أصلاً ولم يحتج أي تعديل، فالمشكلة عابرة تتعلق بحالة الحاوية لا بالكود. لذا اكتفينا بإعادة جمع العينات المتأثرة: أزلنا صفوف P016–P030 من السجل، ونقلنا ملفات JSON القديمة جانباً (لأن نص التصدير يتخطى أي جولة لها ملف موجود مسبقاً)، ثم أعدنا التنفيذ والتصدير من جديد. وعادت أعداد الأحداث إلى نطاقها الصحي (بين ١٧ و ٣٣ حدثاً لكل جولة)، بمحاولة دخول ناجحة وسلسلة أوامر مسجلة في كل منها.

٤.٢ إعادة جمع فئة التخمين عبر الإنترنت

عند مراجعة فئة التخمين عبر الإنترنت اكتشفنا ثغرة منهجية: عدد من الجولات الأصلية استهدف خدمات لا تلتقط الـ honeypots الخاصة بها محاولات الدخول فعلياً — مثل RDP (عبر RDPPhoneypot) وMySQL وMSSQL (عبر Dionaea) — فلم يُسجل عنها سوى أحداث على مستوى الشبكة (P0f وFatt وSuricata) دون أي إشارة دخول. كما كانت جولة Telnet (وهي B029) غير مستقرة وانتهت بمهلة زمنية (timeout) دون تسجيل أي محاولة دخول إطلاقاً.

وبما أن ما يُعرّف فئة التخمين عبر الإنترنت هو سلوك «الفشل ثم النجاح أو الفشل فقط» في الدخول، فإن جولات بلا محاولات دخول لا تمثل الفئة. لذلك أعدنا جمع الفئة بالكامل مستهدفين الخدمات التي تسجل محاولات دخول حقيقية: SSH (عبر Cowrie) وFTP (عبر Dionaea) وVNC (عبر Heralding). وبهذا أصبحت كل عينة في الفئة تحمل السلوك التمييزي المطلوب.

٤.٣ تصحيح عدّ محاولات الدخول (`n_auth_other`)

تُعَدّ سمة `n_auth_other` محاولات الدخول على الخدمات غير Cowrie (أي FTP عبر Dionaea وVNC عبر Heralding). وقد أظهر تعريفها الأول خللاً: فبعض الأحداث تحمل اسم مستخدم مع كلمة مرور فارغة (وهي في الحقيقة مصادفة اتصال لا محاولة دخول)، فكانت تُحسب خطأً وتضخم القيمة. صحّحنا التعريف بأن اشترطنا كلمة مرور غير فارغة لاحتساب المحاولة، فأصبحت محاولات VNC وFTP الحقيقية تُعدّ، بينما تُستبعد مصادفات الاتصال الفارغة. وهذه سمة تغذي لاحقاً سمات سلوك الدخول، فصحتنا مهمة.

٤.٤ التعامل مع التسجيل المتأخر في حجب الخدمة

في فيضانات TCP نصف المفتوحة (SYN وACK وFIN وXMAS وRST)، تُنشئ الحزم تدفقات «بلا جلسة» يسجلها Suricata عند انتهاء مهلتها متأخرة بنحو ٦٠ إلى ١١٠ ثوانٍ عن انتهاء الهجوم الفعلي. ومع فترة تصريف (drain) قصيرة، كانت هذه الأحداث تتسرّب إلى نافذة الجولة التالية؛ فمثلاً ظهرت إحدى جولات فيضان ACK وكأنّها حدث واحد فقط، بينما هبطت إشارتها الحقيقية (نحو ١٦,٨٠٠ حدث) في نافذة الجولة التي تلتها.

حللنا ذلك بإدماج فترة تصريف متدرجة داخل نافذة كل جولة بحسب نوعها: ١٥٠ ثانية لفيضانات TCP بلا جلسة، و٤٥ ثانية لفيضانات UDP وهجمات الطبقة السابعة (L7) التي تُسجل فوراً، و٦٠ ثانية لهجمات الحجب البطيئة. وبهذا التصريف اختفى التسرّب بين الجولات، وأصبح كل حدثٍ منسوباً إلى جولته الصحيحة.

٥. استخراج السمات وبناء مجموعتي البيانات

بعد أن أصبحت لدينا أحداث JSON نظيفة لكلّ جولة، انتقلنا إلى تحويلها إلى صفوف رقمية قابلة للتعلّم. يتولّى هذا الدور النصّ البرمجيّ `extract_features.py`، الذي يقرأ ملفّ كلّ جولة ويحوّله إلى صفّ واحد من السمات (features) الرقمية، ويعيد كتابة مجموعتيّ البيانات كاملتين في كلّ تشغيل.

٥.١ فلسفة السمات: السلوك لا المستشعر

بنينا مجموعتيّ بيانات (datasets) متوازيتين عمداً، لاختبر مبدأ تصميمياً مهماً في المشروع:

- `dataset_behavioral.csv`: سمات سلوكية فقط، دون أيّ إشارة إلى الـ honeypot الذي التقط الأحداث.
- `dataset_full.csv`: السمات السلوكية نفسها، مضافاً إليها ٩ عدادات (counts) لعدد الأحداث الصادرة عن كلّ honeypot على حدة (`n_suricata`, `n_tanner`, `n_dionaea`, `n_cowrie`, `n_p0f`... وهكذا).

الفرق بين المجموعتين هو جوهر التجربة: إن أعطينا النموذج «مِنَ التّقط الحدث» صراحةً (المجموعة الكاملة)، فهل يتحسّن تصنيفه مقارنةً بالاعتماد على السلوك وحده؟ إجابة هذا السؤال نعرضها في القسم التالي.

٥.٢ السمات السلوكية الاثنتان والثلاثون

يُنتِج المستخرج أربعاً وثلاثين سمة، أسقطت منها سمتان ثابتتان (constant) لا تحمّلان أيّ معلومة (`n_file_downloads` و `any_file_download`)، فبقيت ٣٢ سمة فعلية للنموذج. وتتنظّم هذه السمات في ستّ عائلات:

1. **التوقيت والإيقاع (timing):** مثل `total_events` و `duration_sec` و `events_per_sec` و `burst_max_per_sec` و `burstiness` — وهي تلتقط سرعة الهجوم ومدى «انفجاريّته» (`burstiness`)، فتميّز الفيضان السريع من المسح المتمعّل.
2. **المنافذ (ports):** مثل `distinct_dest_ports` و `ports_per_event` و `frac_on_top_port` و `distinct_protocols` — وتلتقط ما إذا كان الهجوم يجتاح منافذ كثيرة (مسح) أم يتركز على منفذ واحد (تخمين عبر الإنترنت أو حجب خدمة).
3. **الدخول والمصادقة (login/auth):** مثل `n_login_fail` و `n_login_success` و `n_auth_other` و `any_login_attempt` — وتلتقط سلوك «الفشل ثمّ النجاح» المميّز للتخمين عبر الإنترنت.
4. **الأوامر والجلسات (commands/sessions):** مثل `n_commands` و `n_command_failed` و `n_sessions` و `distinct_commands` — وتلتقط نشاط الصدفة (shell) المميّز لما بعد الاختراق.
5. **الويب (web):** مثل `n_http_events` و `n_web_80` و `n_web_8080` و `distinct_web_paths` — وتلتقط نمط طلبات هجوم الويب.

6. تنبيهات Suricata وتدفقاته (network): مثل n_suricata_alerts و n_suricata_flows و n_alert_sev_high و distinct_signatures — وتضيف رؤية على مستوى الشبكة مستقلة عن أي honeypot.

٥.٣ ضمانات منع تسريب المعلومات (leakage safeguards)

حرصنا على ألا «يغش» النموذج بالاعتماد على معلومات تكشف الإجابة أو تخص بيئتنا لا الهجوم نفسه. لذلك استبعدنا عمداً:

- العناوين المنطقية والموقع الجغرافي (IP / geoip): فهي ثابتة في مختبرنا (كل حدث من 192.168.32.1) ولا تحمل معلومة تمييزية.
 - أرقام المنافذ المصدرية: لأنها عشوائية عابرة (ولقد أبقينا على * عددها* بوصفه مؤشراً لعدد الاتصالات).
 - محتوى الحمولات والبصمات ومعرفات الجلسات (hashes / session UUIDs): لأنها تدفع النموذج إلى الحفظ (memorization) بدل التعلم.
 - وقت اليوم (time-of-day): لأنه أثر لجدول عملنا لا خاصية للهجوم.
 - حقل تصنيف Tanner (detection.name): وهو الأهم، إذ يمثل خط الأساس القائم على القواعد (rule-based baseline) الذي نقارن نموذجنا به؛ فإدخاله سمة يعني تسريب الإجابة إلى النموذج.
- بنهاية هذه المرحلة أصبحت لدينا مجموعتان جاهزتان: ١٥٠ صفًا (جولة) لكل منهما، إحداها بـ ٣٢ سمة والأخرى بـ ٤١ سمة، وكلتاها خالية من الخلايا الفارغة وجاهزة للتدريب.

٦. تدريب النماذج ومقارنتها

في هذه المرحلة انتقلنا من البيانات إلى النماذج. نفّذنا العمل عبر ثلاثة دفاتر Jupyter متتابعة: الأول لاستكشاف البيانات وإثبات الفرضية أن النموذج المدرب على البيانات السلوكية ستكون دقته قريبة من النموذج المدرب على كامل البيانات (والتي تحتوي مؤشرات عن الـhoneypots)، والثاني لضبط نموذج الغابة العشوائية (Random Forest)، والثالث لمقارنة ستة نماذج مختلفة واختيار الأفضل للنشر.

٦.١ بروتوكول التقييم

نظراً لصغر حجم مجموعة البيانات (١٥٠ عينة)، لم نقطع منها مجموعة اختبار منفصلة (hold-out)، لأن حجب جزء من البيانات كان سيكلفنا دقة دون مقابل. اعتمدنا بدلاً من ذلك التحقق المتقاطع الطبقي خماسي الطيات (fold-5 stratified cross-validation)، الذي يقسم البيانات إلى خمس طيات متوازنة، فيدرب على أربع ويختبر على الخامسة بالتناوب، لنحصل على تقدير أمتن لأداء «الوصفة» كلها. أما النموذج المنشور نهائياً فيدرب على العينات الـ ١٥٠ كاملة، ويكون الرقم المبلّغ عنه (المقيس على طيات أصغر) تقديراً لأدائه.

ونشير إلى أن الدقة (accuracy) مقياس عادل هنا لأن الفئات متوازنة تماماً (٣٠ عينة لكلٍ منها)؛ فنموذج كسول يخمن فئة واحدة دائماً لا يتجاوز ٢٠٪. ولهذا جاءت الدقة مساوية لمقياس macro-F1 في نتائجنا، وهو دليل على أن الأداء متنسق عبر الفئات لا منحاز لإحداها.

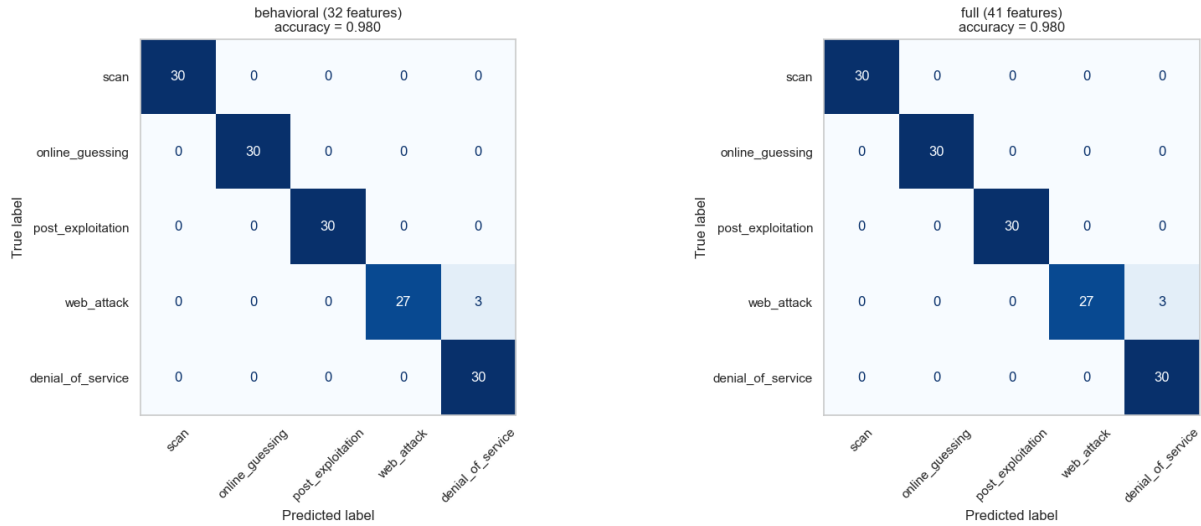
على مستوى المعالجة، لم نطبق أيّ تحجيم (scaling) على النماذج الشجرية لأنها غير حسّاسة للمقياس أصلاً، بينما أدرجنا مُحجّماً قياسيًّا (StandardScaler) داخل خطّ معالجة (Pipeline) للنماذج الحسّاسة للمسافات والتدرّجات (SVM و KNN والانحدار اللوجستي والشبكة العصبية)، بحيث يتعلّم المُحجّم من طيّات التدريب وحدها منعاً لأيّ تسريب. كما رُمّزت الوسوم إلى أعداد صحيحة من 0 إلى 4 بترتيب ثابت، لأنّ XGBoost يتطلّب ذلك، ولتبقى مصفوفة الالتباس (confusion matrix) بترتيب واحد في كلّ النماذج.

٦.٢ إثبات الفرضية: السلوك لا المستشعر

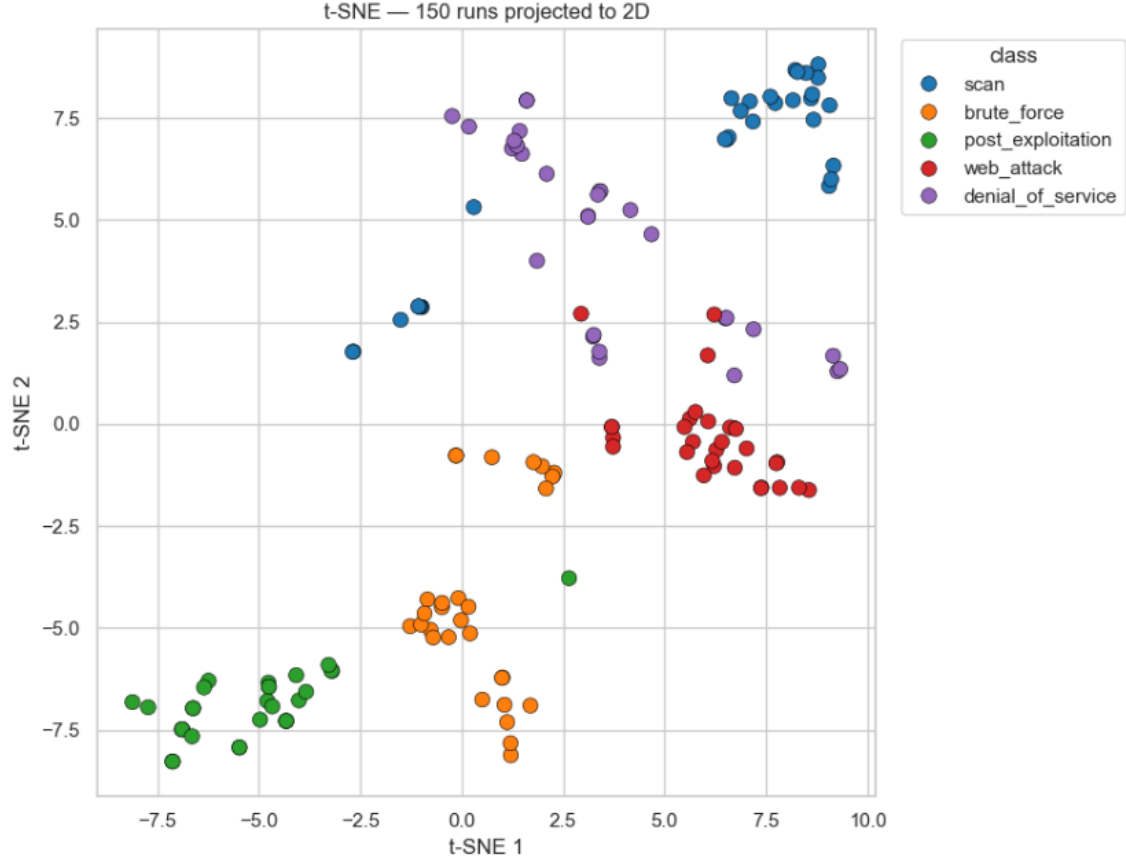
بدأ الدفتر الأوّل باستكشاف البيانات بصرياً عبر مخطّطات الصناديق (boxplots) وخريطة الارتباط (correlation heatmap) وتخفيض الأبعاد بطريقتيّ PCA و t-SNE. وكشف الاستكشاف أنّ سمات الحجم تتوزّع على مقاييس شديدة التفاوت مع انحرافٍ يمينيٍّ قويٍّ (المتوسّط أكبر بكثير من الوسيط)، وهو ما يؤكّد ملائمة النماذج الشجرية (التي تقسّم بالعتبات ولا تتأثّر بالمقياس) ويبرّر عدم الحاجة إلى التحجيم معها.

ثمّ أجرينا تجربةً مفصلية: درّبنا غابةً عشوائيةً على المجموعتين — السلوكية (٣٢ سمة) والكاملة (٤١ سمة) — وقارنّا النتيجة. جاءت الدقّة متطابقةً تماماً: 0.9800 ± 0.0163 في كلتا الحالتين، بل كانت تنبؤات النموذجين متطابقةً على العينات الـ ١٥٠. ومعنى ذلك أنّ إضافة عدّادات «مَن التقط الحدث» لم تُضِف شيئاً: النموذج يصنّف الهجوم بحسب سلوكه لا بحسب المستشعر الذي رصده. وبذلك تأكّد هذا المبدأ، واعتمدنا المجموعة السلوكية أساساً للنشر.

أنتج هذا النموذج ثلاثة أخطاء فقط: العينات W001 و W002 و W003 (هجمات ويب عالية الحجم تشبه فيضانات حجب الخدمة).



الشكل 20 مصفوفة الالتباس للغابة العشوائية التي تظهر نفس الأخطاء لنموذج مدرب على كامل السمات أو على السلوكية فقط.



الشكل 21: مخطّط t-SNE يُظهر تجمّع الفئات الخمس وانفصالها في فضاء السمات.

٦.٣ ضبط الغابة العشوائية

في الدفتر الثاني ضبطنا مُعاملات الغابة العشوائية عبر بحثٍ شبكيّ واسع (GridSearch). وكانت الدقّة $\pm 98,0\%$ ، لكنّ الملاحظة الأهمّ أنّ أفضل ستّين تركيباً من المُعاملات جاءت متعادلةً عند القيمة نفسها تقريباً — أي هضبةً مسطّحةً (flat plateau) لا ترتيّبُ حادّ. ومعنى ذلك أنّ الفئات منفصلةٌ بما يكفي لجعل الضبط الدقيق للمُعاملات قليل الأثر.

٦.٤ مقارنة ستّة نماذج

في الدفتر الثالث وضعنا ستّة نماذج على قدم المساواة ضمن إطار تقييمٍ موحدٍ (بضبطٍ شبكيّ لكلّ نموذج، وبالتحجيم للنماذج التي تحتاجه دون الشجرية). أعدنا تسجيل الغابة العشوائية ضمن الإطار نفسه فأعطت النتيجة عينها ($98,0\%$) تأكيداً لنتيجة الدفتر الثاني، ثمّ درّبنا البقية. يلخّص الجدول التالي النتائج:

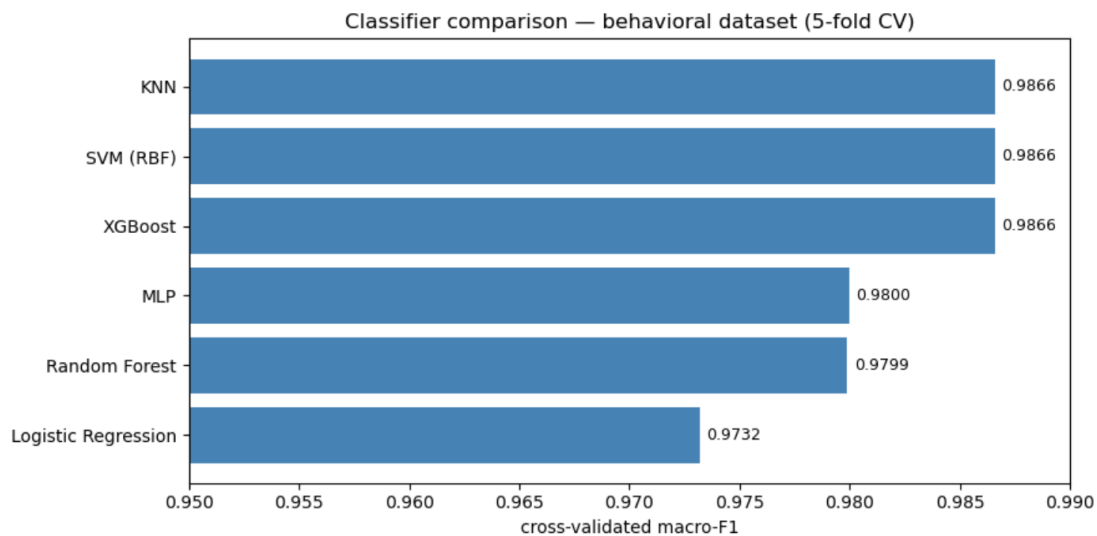
الجدول 6: مقارنة النماذج الستّة بالتحقق المتقاطع الطبقي (5 طيّات).

النموذج	الدقّة (accuracy)	macro-F1	الأخطاء	محجّم؟
XGBoost	98,67%	98,66%	٢	لا
SVM (خطّي)	98,67%	98,66%	٢	نعم
KNN	98,67%	98,66%	٢	نعم

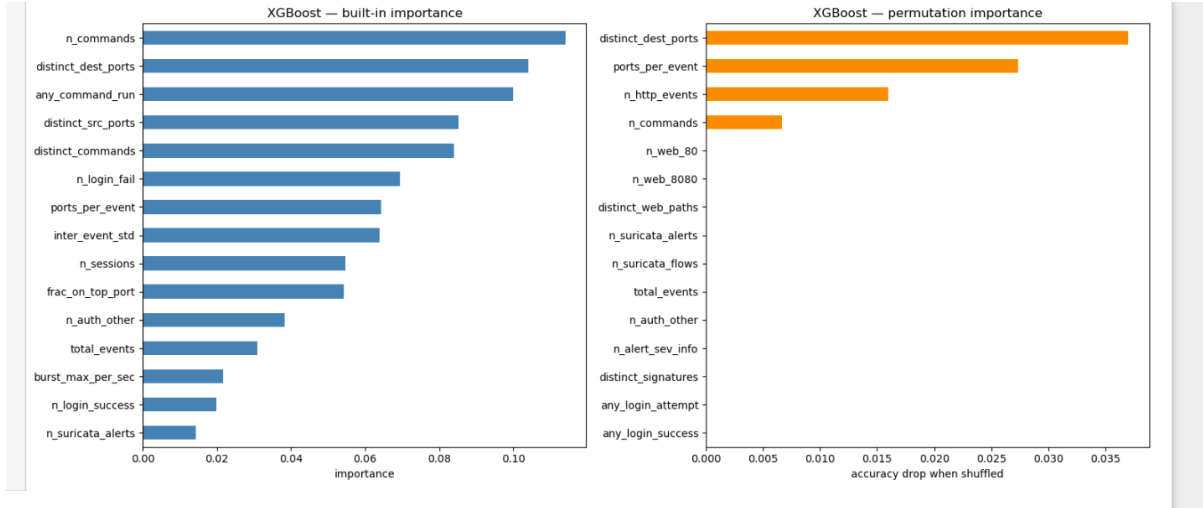
النموذج	الدقة (accuracy)	macro-F1	الأخطاء	مجمّم؟
Random Forest	٪٩٨,٠٠	٪٩٧,٩٩	٣	لا
MLP	٪٩٨,٠٠	٪٩٨,٠٠	٣	نعم
Logistic Regression	٪٩٧,٣٣	٪٩٧,٣٢	٤	نعم

تصدّرت ثلاثة نماذج القائمة متعادلةً عند ٪٩٨,٦٧ بخطأين فقط: XGBoost و SVM الخطّي و KNN. ومن أطرف ما ظهر أنّ نواة SVM الفائزة كانت **خطيّة (linear)**، بينما تأخّرت النواة غير الخطيّة (RBF) — ما يعني أنّ الفئات **قابلة للفصل خطيّاً** في فضاء السمات الاثنتين والثلاثين، وأنّ النواة الأعقد لم تفعل سوى المبالغة في الملاءمة (overfitting). غير أنّ نموذجاً خطيّاً آخر هو الانحدار اللوجستي جاء أدنى (٪٩٧,٣)؛ فالفئات إذن **شبه قابلة للفصل خطيّاً**، ومن بين الطرائق الخطيّة يتفوّق SVM القائم على تعظيم الهامش (margin) على الانحدار اللوجستي القائم على الاحتمالات.

ومن اللافت أيضاً اختلاف «شكل» اعتماد النماذج على السمات: يوزّع XGBoost اعتماده على سمات سلوكية كثيرة (any_command_run, ports_per_event, distinct_dest_ports, n_commands, ...distinct_commands)، بينما تتكئ الغابة العشوائية على سمة مهيمنة واحدة تقريباً هي any_login_attempt.



الشكل 22: الرسم البياني لمقارنة النماذج الستة بحسب الدقة.



الشكل 23: أهمية السمات (feature importance) في نموذج XGBoost المنشور.

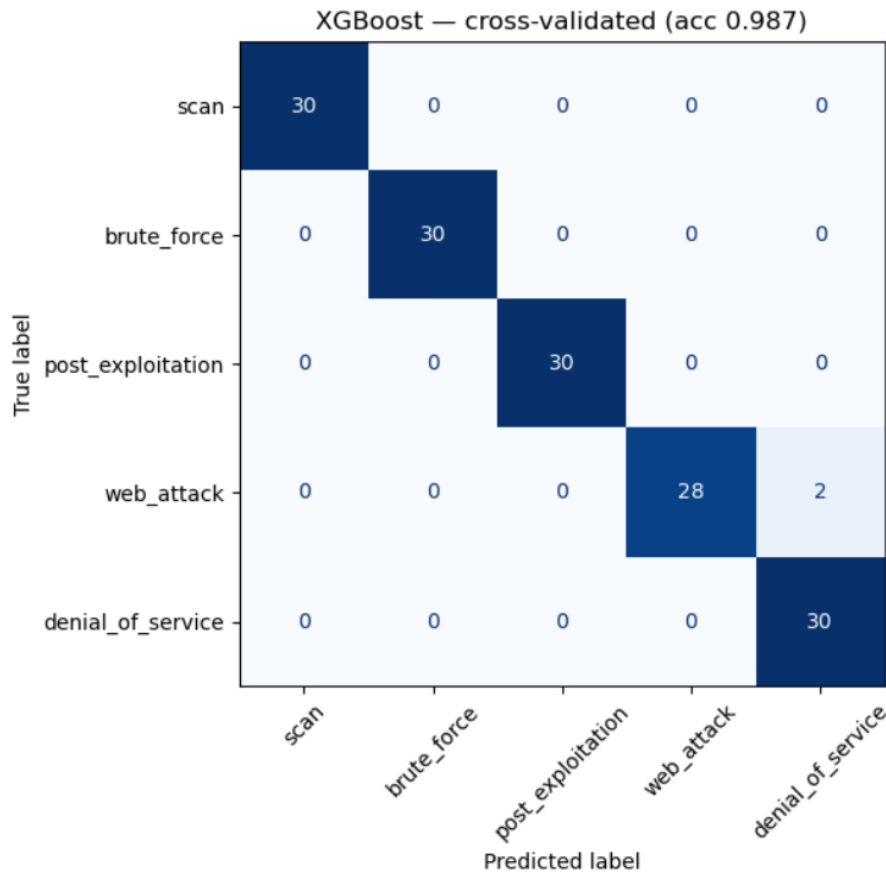
٦.٥ مجموعة الأخطاء الموحدة

هنا برزت أهم نتيجة تحليلية في المشروع. فعند جمع أخطاء النماذج الستة جميعها، تبين أن كل حالة تصنيف خاطئ — دون استثناء — تقع ضمن مجموعة صغيرة واحدة:

{ W001، W002، W003، D020، D027، P015 }

أي أن أي عينة خارج هذه المجموعة لم يُخطئ فيها أي نموذج قط. فئات المسح والتخمين عبر الإنترنت وما بعد الاختراق صُنِّفت تصنيفاً يكاد يكون تاماً في كل النماذج (باستثناء العينة P015 الشاذة وحدها). أما سطح الخطأ بأكمله فيتألف من أمرين: (أ) الحدّ الفاصل بين الويب وحجب الخدمة من حيث الحجم، وهو التباس في الاتجاهين معاً (النماذج الشجرية تميل إلى «ويب ← حجب خدمة»، بينما SVM والانحدار اللوجستي يميلان إلى «حجب خدمة ← ويب»، وKNN يقسم واحدة لكل)، و(ب) العينة P015، وهي جولة ما بعد اختراق فعلاً على الحدود (أخطأت فيها SVM والانحدار اللوجستي والشبكة العصبية بشكل مستقل).

وتفسير ذلك أن التباس الويب وحجب الخدمة ضبابية حقيقية ثنائية الاتجاه: فجولات الويب الضخمة (نحو ٢٤ ألفاً إلى ٣٨ ألف حدث) تشبه الفيضانات فعلاً. والأهم أن هذا الالتباس ظهر عبر ثلاث آليات تعلّم مختلفة تماماً (تقسيمات الأشجار، والمستوى الفائق الخطّي، والتصويت بالمسافة).



الشكل 24: مصفوفة الالتباس في نموذج XGBoost المنشور.

٧. نشر النموذج والتصنيف الحي

بعد اختيار النموذج، انتقلنا إلى تشغيله عملياً: حفظه بصيغة قابلة للنقل، وربطه بخط يسحب البيانات الحية ويصنفها، ثم بناء مكون يعمل بشكل مستمر.

٧.١ النموذج المنشور والملفات المحفوظة

اخترنا نشر **XGBoost**، فهو من النماذج الثلاثة المتصدرة (٩٨,٦٧٪ بخطأين)، وقد استعاد العينة W002 التي أخطأت فيها النماذج الشجرية.

أعدنا تدريب التركيب الفائز على العينات الـ ١٥٠ كاملة، وحفظنا النموذج والملفات المرافقة في مجلد 06-models/:

- **deployed_model.json**: النموذج بصيغة XGBoost الأصلية، وهي صيغة قابلة للنقل (portable) تُحمّل عبر مختلف الإصدارات.
- **deployed_model.pkl**: النموذج بصيغة joblib (أسرع تحميلاً لكنها أكثر حساسية لاختلاف الإصدارات).
- **feature_columns.json**: أسماء السمات الاثنتين والثلاثين بترتيبها الدقيق.
- **class_labels.json**: جدول تحويل الرقم الممتنّب به إلى اسم الفئة.
- **model_comparison.csv** و **deployed_model_meta.json**: سجلّ المقارنة والبيانات الوصفية (المعاملات، ودرجات التحقق، وإصدارات المكتبات، والطابع الزمني) لأغراض إعادة الإنتاج.

ولضمان أن تعمل النسخة المحفوظة على آلة الهجوم أيضاً، طابقنا إصدارات المكتبات بين البيئتين (xgboost و 3.2.0 و sklearn 1.9.0).

٧.٢ معمارية التصنيف والربط بـ Kibana

من المهمّ التوضيح أنّ Kibana لا تشغّل النماذج، بل تعرض فهرساً (index) في Elasticsearch وحسب. لذلك يجري التصنيف في Python لا داخل Kibana، وفق المعمارية التالية:

1. يحمّل برنامج Python النموذج ووحدة extract_features.py نفسها (دون تكرار منطق السمات).
2. يسحب نافذةً حيّةً من الأحداث من Elasticsearch، ويبنى منها متّجه السمات، ويتنبّأ بالفئة.
3. يكتب وثيقة تنبؤ بصيغة JSON إلى فهرس جديد (مثل ml-predictions-*) .
4. تقرأ لوحات Kibana هذا الفهرس وتعرض النتائج.

أي أنّ ما يصل إلى Kibana هو وثائق تنبؤ بصيغة JSON. وقد تحقّقنا من سلامة هذا المسار على بيانات حقيقية، فأعطى تنبؤات صحيحة لكلّ فئة (مثالاً: تخمين عبر الإنترنت ٩٦,٧٪، حجب خدمة ٩٩,٣٪، ما بعد اختراق ٩٨,٦٪، مسح ٩٨,١٪، هجوم ويب ٨٩,٤٪).

٧.٣ التصنيف الحي: كشف الجولات المحدود بالخمول

بدأنا بمصنّف يعمل «بالطلقة الواحدة» (live_classify.py) يسحب نافذةً عبر Scroll API بالمرشّح المصدريّ نفسه، ويبنى المتّجه بترتيب السمات المحفوظ، ويتنبّأ. ثمّ طوّرناه إلى المكوّن الحيّ المعتمد live_runs.py، الذي يقوم على فكرة كشف الجولات المحدود بالخمول (idle-bounded run) (detection): يعامل التسلسل «خمول ← نشاط ← خمول» بوصفه جولة هجوم كاملة، فحين تمرّ مدّة محدّدة (idle) دون أحداث جديدة، يسحب كلّ أحداث الجولة ويصنّفها دفعةً واحدة. وميزة هذا الأسلوب أنّه يطابق الطريقة التي تُربّ بها النموذج (على جولات كاملة)، ويفصل الجولات المتتابعة لمهاجم واحد فصلاً نظيفاً دون الحاجة إلى نافذة منزلق (sliding window) ثابتة. ضبطنا فجوة الخمول بحسب طبيعة كلّ فئة:

- ٣٠ ثانية (الافتراضي) للمسح وهجوم الويب والتخمين عبر الإنترنت وما بعد الاختراق، إذ تكتمل ذروتها المعنوية خلال نحو عشر ثوانٍ.
 - ١٢٠ ثانية لحجب الخدمة، لأنّ أحداث تدفق Suricata المميّزة له تصل متأخّرةً حتى نحو ٨٧ ثانية (وقد ضُمّنت في تدريبه عبر فترة التصريف).
- وأضفنا حدّاً أقصى لطول الجولة (٩٠٠ ثانية افتراضياً) يفرض تصنيف أيّ جولة لا تهدأ إطلاقاً (كفيضانٍ متواصل). وفي العرض الحيّ، صنّف المكوّن فيضان حجب خدمة استمرّ ٨١ ثانية بثقة 98.8٪، معتمداً على سماتٍ دالة واضحة: نحو 27226 حدث، ومنفذ وجهة واحد، وصفرٌ في محاولات الدخول والأوامر وطلبات الويب.

```

(hsen@ Kali)-[/mnt/c/tpot-project/03-scripts]
$ python3 live_runs.py --idle 120
loading deployed model...
model loaded | poll=4s idle-gap=120s min-events=10 max-run=900s

watching the honeypot – run attacks one at a time, spaced > idle-gap apart.
Ctrl+C to stop.

· idle – waiting for an attack...
▶ attack detected 22:03:13 – collecting...
  collecting: 2848 events | run 200s | idle 119s / 120s

=====
ATTACK RUN CLASSIFIED (22:03:13 -> 22:04:33, 81s)
=====
>>> DENIAL_OF_SERVICE (confidence 98.8%)

denial_of_service 98.8% #####
web_attack        0.6%
scan              0.2%
post_exploitation 0.2%
online_guessing   0.2%

key behavior:
total_events      = 27226
distinct_dest_ports = 1
n_login_fail      = 0
n_login_success   = 0
n_commands        = 0
n_http_events     = 0
n_web_80          = 27226
n_web_8080        = 0

✓ written to dashboard index 'ml-predictions'
· idle – waiting for an attack...   ting for an attack...

```

الشكل 25 خرج live_runs.py وهو يصنّف جولة حجب خدمة حيّة مع درجات الثقة والسمات الدالة.

٨. لوحة معلومات Kibana الحيّة لعرض التصنيف

بعد أن أصبح المصنّف الحيّ يعمل ويطلع نتيجته في الطرفية (terminal)، أضفنا طبقة العرض: خزناً كلّ تنبؤ (prediction) في Elasticsearch، ثمّ بنينا لوحة معلومات (dashboard) في Kibana تعرضه بصرياً وتتحدّث حيّاً مع كلّ هجوم. والمبدأ الحاكم امتداداً لما سبق: **Kibana لا تشغّل النموذج، بل Python هو الذي يشغله؛** فوظيفة Kibana أن تقرأ فهرساً (index) وترسمه لا أكثر.

٨.١ الهدف ومسار البيانات والقرار المعماري

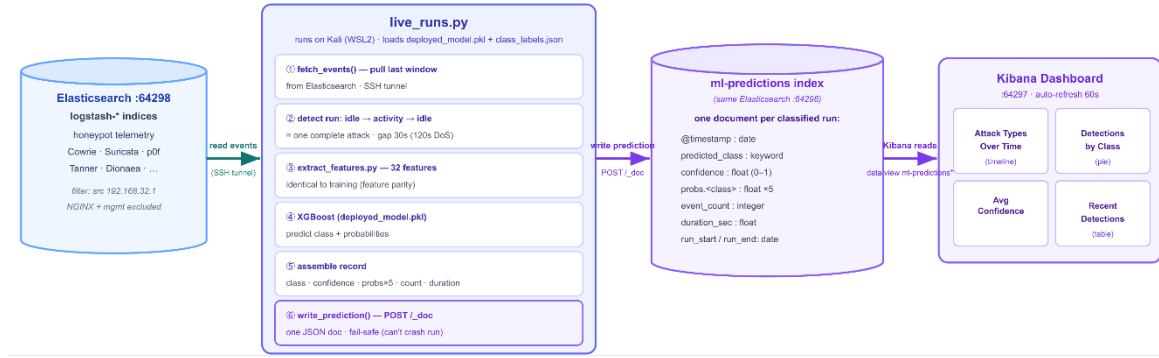
الهدف المباشر: أخذ التنبؤات التي يُنتجها live_runs.py أصلاً وتخزينها في Elasticsearch كي تتمكّن Kibana من رسمها. ومسار البيانات: يكتشف live_runs.py جولة هجوم ويصنّفها (وهذا يعمل من المرحلة السابقة)، ثمّ — والإضافة الجديدة هنا — بدل أن يطبع النتيجة فقط، يكتبها أيضاً وثيقة (document) في فهرس جديد اسمه ml-predictions، فتقرأ لوحة Kibana ذلك الفهرس وترسمه. وينقسم العمل إلى ثلاث خطوات: كتابة

التنبؤات في فهرس، ثم تسجيل الفهرس في Kibana بوصفه «عرض بيانات» (data view)، ثم بناء اللوحة (أنواع الهجوم عبر الزمن، وتوزيع الفئات، ومتوسط الثقة، وجدول آخر الاكتشافات).

وقد اخترنا دمج الكتابة داخل `live_runs.py` نفسه (فيكتب المصنّف إلى الفهرس أثناء عمله، فتحدّث اللوحة مع كلّ هجوم في العملية نفسها).

Architectural Schematic — Live Classification → ml-predictions → Kibana Dashboard

`live_runs.py` classifies each attack run and writes it to Elasticsearch; Kibana reads the ml-predictions index and visualizes it.



Both indices live in the same Elasticsearch (64298), reached from Kali through the SSH tunnel: 64295 → localhost:64298.

The dashboard write is fail-safe — a failure prints a warning but never stops classification. teal = read via tunnel - violet = prediction write + visualize.

الشكل 26: مخطط المعمارية: `live_runs.py` يصنّف الجولة ويكتبها في فهرس `ml-predictions`، ثم تقرأ لوحة Kibana وترسمه.

٨.٢ اختبار إمكانية الكتابة إلى Elasticsearch

قراءة البيانات من هذا الـ Elasticsearch تعمل أصلاً، لكن الكتابة صلاحية مختلفة تماماً ولها أعطال مختلفة. لذلك، وقبل المساس بالمصنّف العامل، تحقّقنا من الكتابة على فهرس مؤقت. وأخذنا في الحساب عدّة أعطال محتملة: أن تسمح المنظومة بالقراءة دون مصادقة (anonymous) وتمنع الكتابة فتعود بخطأ 403؛ وأن تمنع سياسة الإنشاء التلقائي للفهرس الكتابة إلى فهرس جديد (نتجاوزها بإنشاء الفهرس صراحةً عبر PUT)؛ وأن يكون العنقود (cluster) للقراءة فقط بسبب امتلاء القرص (فتفشل الكتابة)؛ وأن يكون تخطيط الحقول (mapping) غير ملائم لـ Kibana. كما اعتمدنا قاعدة واحدة: نُخزّن درجة الثقة وكلّ احتمالات الفئات كسوراً في المجال [0,1] (الخرج الخام للنموذج)، وتتولّى Kibana عرضها نسباً مئوية.

كتبنا نصّاً برمجياً يُنشئ فهرساً مؤقتاً للتجربة (اختير عمداً كي لا يطابق نمط `ml-predictions*`)، ويكتب فيه وثيقة، ويقرأها، ويعرض تخطيطها، ثم يحذف نفسه. وجاءت النتائج كلّها خضراء: الوصول HTTP 200 (النسخة 9.3.5)، وإنشاء الفهرس HTTP 200 (acknowledged)، وكتابة الوثيقة HTTP 201 (created) دون أي خطأ 403، وقراءتها ثانيةً بنجاح، وتخطيط جاهر لـ Kibana (الطابع الزمني date، والفئة keyword، والاحتمالات float)، ثم حذف الفهرس المؤقت. خلاصة: الكتابة تعمل دون مصادقة، وإنشاء الفهرس صراحةً مسموح، والعنقود قابل للكتابة.

٨.٣ تعديل النصّ البرمجي `live_runs.py` لكتابة التنبؤات

الفهرس الحقيقي هو ml-predictions (فهرس واحد محدّد)، بينما يستعمل عرض البيانات في Kibana النمط ml-predictions* كي يبقى مطابقاً لو جُدد الفهرس. وأدخلت أربعة تغييرات على النصّ البرمجي: (١) إضافة ثابت اسم الفهرس PRED_INDEX؛ (٢) دالة ensure_index() التي تُنشئ الفهرس مرّة واحدة عند البدء بتخطيط صريح، وهي عديمة الأثر (idempotent) لا تفعل شيئاً إن كان الفهرس موجوداً؛ (٣) دالة write_prediction() التي تُرسل وثيقة واحدة بعد كلّ تصنيف؛ (٤) استدعاء الكاتب داخل classify_run() عقب طباعة الحكم، مع إنشاء الفهرس عند تحميل النموذج.

نُشئ الفهرس صراحةً بدل ترك Elasticsearch يُخمن أنواع الحقول: الطابع الزمني @timestamp من نوع date، والفئة predicted_class من نوع keyword، والثقة والمدة وعدّاد الأحداث والاحتمالات أرقام. وتحمل كلّ وثيقة: الطابع الزمني، والفئة المتنبأ بها، ودرجة الثقة، وعدد الأحداث، والمدة بالثواني، ووقتي بداية الجولة ونهايتها، واحتمالات الفئات الخمس كاملة.

أما أهم قرار فهو تصميم الأمان: كلتا الدالتين مغلفتان بمعالجة أخطاء (try/except) تكفي بطباعة تحذير، فلا يمكنهما إسقاط المصنّف أبداً. فالمبدأ: التصنيف هو المنتج، واللوحة إضافة مكتملة.

٨.٤ التحقق الشامل من طرف إلى طرف

بالنق مفتوحاً، شغلنا المصنّف في طرفية أولى، وأطلقنا مسحاً (scan) سريعاً واحداً من طرفية ثانية، ثم قرأنا الفهرس. في الطرفية، أنشأت ensure_index() الفهرس مرّة واحدة (HTTP 200)، ثم اكتشيف الهجوم وصنّف على أنّه المسح (scan) بثقة 98.9% مع distinct_dest_ports = 100 (أي nmap يطرق مئة منفذ)، وظهرت رسالة نجاح الكتابة. وعند قراءة الفهرس، وجدنا الوثيقة مخزّنة بالشكل الصحيح تماماً: الفئة scan، ودرجة الثقة 0.989 (كسر كما خططنا)، وعدد الأحداث 204، والمدة 71 ثانية، مع تفصيل الاحتمالات.

```

(hsen@ Kali)-[/mnt/c/tpot-project/03-scripts]
$ python3 live_runs.py --idle 120
loading deployed model...
model loaded | poll=4s idle-gap=120s min-events=10 max-run=900s

watching the honeypot – run attacks one at a time, spaced > idle-gap apart.
Ctrl+C to stop.

• idle – waiting for an attack...
▶ attack detected 22:16:02 – collecting...
  collecting: 102 events | run 188s | idle 117s / 120s

=====
ATTACK RUN CLASSIFIED (22:16:02 -> 22:17:13, 71s)
=====
>>> SCAN (confidence 98.9%)

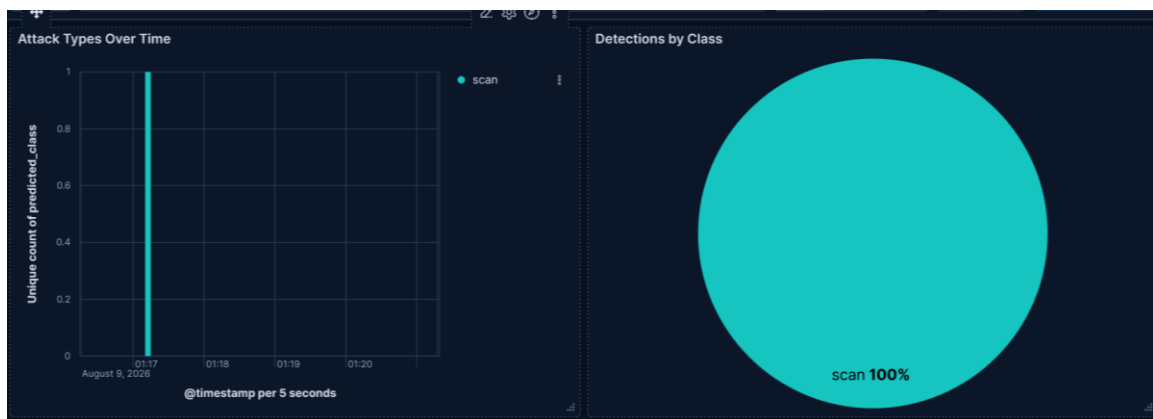
scan          98.9% #####
online_guessing 0.4%
post_exploitation 0.3%
denial_of_service 0.2%
web_attack     0.2%

key behavior:
total_events      = 204
distinct_dest_ports = 100
n_login_fail      = 0
n_login_success   = 0
n_commands        = 0
n_http_events     = 0
n_web_80          = 3
n_web_8080        = 2

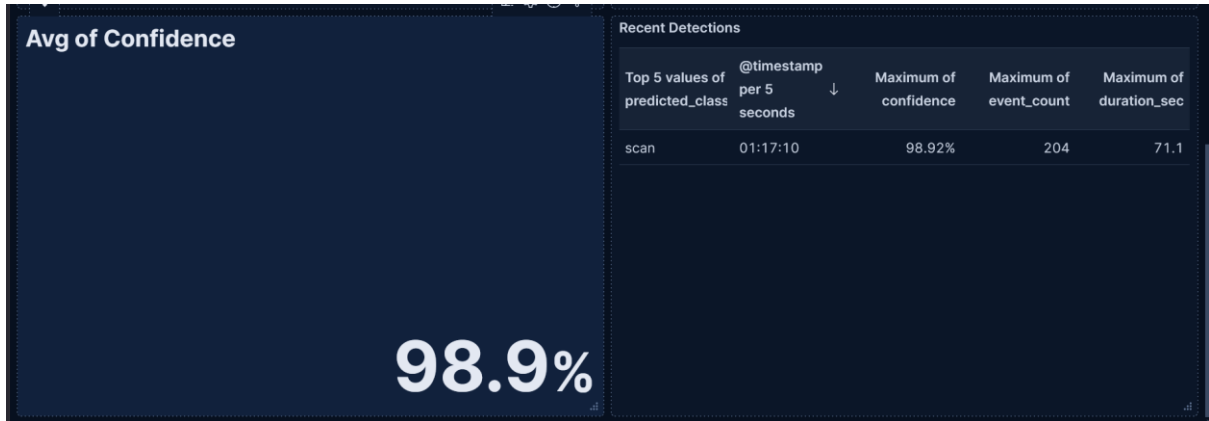
✓ written to dashboard index 'ml-predictions'
• idle – waiting for an attack...   ting for an attack...

```

الشكل 27: خرج طرفية المصنّف الحيّ يُظهر تصنيف المسح.



الشكل 28: أول ظهور للتنبؤ في لوحة Kibana بعد التحقق الشامل: جولة مسح واحدة في الخطّ الزمني وتوزيع الفئات.



الشكل 29 الجزء السفلي من لوحة kibana يعرض المعلومات التي تم تسجيلها عن عملية المسح.

٨.٥ تسجيل عرض البيانات في Kibana

قبل رسم أي شيء، على Kibana أن «تعرف» الفهرس بتسجيله عرض بيانات (data view) مرة واحدة: نفتح Kibana على <https://192.168.32.138:64297>، ثم من

Management → Stack Management → Kibana → Data Views → Create data

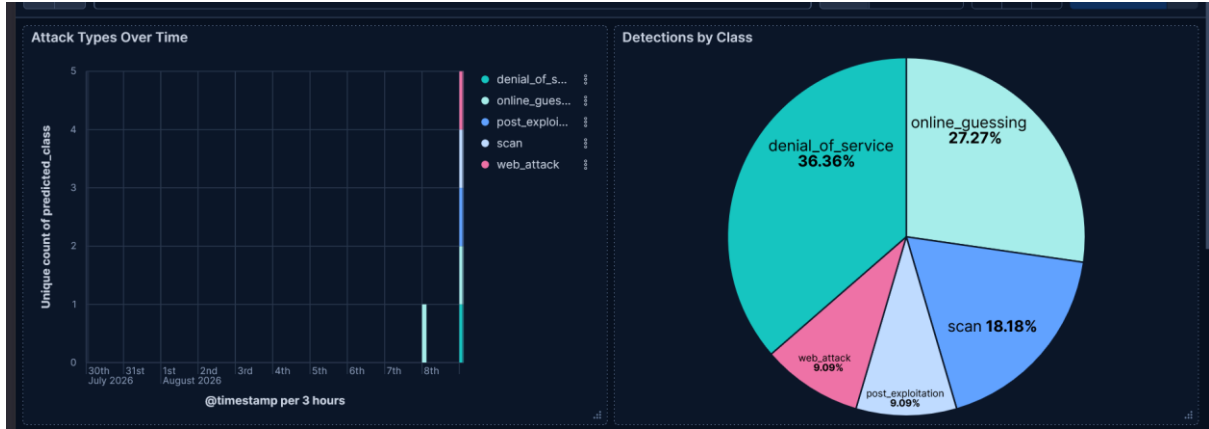
view ندخل الاسم ml-predictions والنمط ml-predictions* وحقل الطابع الزمني @timestamp ونحفظ. وللتأكد، فتحنا **Discover** واخترنا عرض البيانات ومدى «آخر ٢٤ ساعة» فظهرت وثيقة المسح صفًا. وننّبه إلى أن الوثائق مختومة بتوقيت UTC بينما تعرض Kibana بالتوقيت المحلي، فقد تبدو الصفوف مُزاحة بضع ساعات؛ وإن بدا Discover فارغاً يكفي توسيع المدى الزمني.

٨.٦ بناء لوحة المعلومات

من **Dashboard → Create dashboard** فتحنا لوحة في وضع التحرير، وضبطنا المدى على «آخر ٢٤ ساعة»، وفعلنا التحديث التلقائي (auto-refresh) كل 60 ثانية. ولكل لوحة فرعية (panel) أنشأنا رسماً عبر محرر Lens مع التأكد أن عرض البيانات المختار هو ml-predictions. وبنينا أربع لوحات فرعية:

1. أنواع الهجوم عبر الزمن (Attack Types Over Time) — القطعة المركزية: رسم عمودي مكّس (stacked bar)، محوره الأفقي @timestamp كمدرج زمني، والعمودي عدد السجلات، مقسماً بحسب predicted_class — فينتج أعمدة ملونة مكّسة، لون لكل نوع.
2. توزيع الفئات (Detections by Class): رسم دائري بشرائح بحسب predicted_class بحجم يوافق العدد.
3. متوسط الثقة (Avg Confidence): مقياس (metric) قيمته متوسط confidence بصيغة نسبة مئوية.
4. آخر الاكتشافات (Recent Detections): جدول أعمدته الطابع الزمني والفئة والثقة وعدد الأحداث والمدة، مرتباً تنازلياً بحسب @timestamp.

ثم حفظنا اللوحة باسم **"Honeypot Attack Classification — Live"**. وأثناء العرض الحي، تكتب كل نتيجة وثيقة، وخلال نحو عشر ثوانٍ تتحدث اللوحات الأربع: يكبر عمود في الخط الزمني، وتُضاف شريحة إلى الدائرة، ويستقر رقم متوسط الثقة، ويتصدّر صفٌّ جديد الجدول.



الشكل 30: لوحة المعلومات (الجزء الأعلى): أنواع الهجوم عبر الزمن وتوزيع الفئات.



الشكل 31: لوحة المعلومات (الجزء الأسفل): متوسط الثقة وجدول آخر الاكتشافات.

9. خطة الاختبارات

جرى التحقق من النظام على أربعة مستويات: اختبارات الوحدة، واختبارات التكامل، واختبارات الأمان، واختبار التحميل. ونعرض لكل مستوى هدفه وطريقته ونتيجته.

9.1 اختبارات الوحدة (unit tests)

تستهدف التأكد من صحة كل وحدة على حدة:

1. **صحة استخراج السمات:** التحقق من أنّ جولة ما ذات أحداث معلومة تنتج قيم سمات متوقعة (مثل عدّ الأحداث، والمدة، وعدد محاولات الدخول). النتيجة: مطابقة القيم المحسوبة للمتوقع.
2. **تصحيح عدّ محاولات الدخول (n_auth_other):** اختبار أنّ محاولة بكلمة مرور فارغة (مصافحة اتصال) لا تُحسب كمحاولة دخول. النتيجة: بعد اشتراط كلمة مرور غير فارغة، أصبح العدّ صحيحاً.
3. **سلامة الالتقاط لكلّ جولة (عبر verify_*.py):** التأكد من أنّ كلّ جولة التقطت عدداً غير صفري من الأحداث ومن الـ honeypot المتوقع. النتيجة: كلّ الجولات غير صفريّة، ولا نوافذ متداخلة، ولا صفوف مكرّرة.
4. **تحميل النموذج والتنبؤ المفرد:** اختبار أنّ النموذج المحفوظ يُحمّل ويُنتج تنبؤاً واحداً صحيحاً لمتجه سمات معلوم. النتيجة: تنبؤ صحيح مع درجة ثقة.
5. **اختبار الكتابة إلى Elasticsearch (اختبار مبدئي على فهرس مؤقت):** إنشاء الفهرس، وكتابة وثيقة، وقراءتها، والتحقق من تخطيط حقولها، ثم حذفها. النتيجة: نجحت الخطوات كلّها (created / 200 / 201 / تخطيط صحيح / محذوف).

٤.٢ اختبارات التكامل (integration tests)

تستهدف التأكد من صحة الحدود بين الأجزاء:

1. **الخطّ الكامل من طرف إلى طرف:** التحقق من تطابق «شكل» البيانات عند كلّ حدّ — من الأحداث الخام، إلى ملفات JSON، إلى صفوف السمات، إلى التنبؤ. النتيجة: تدفق متسق دون فقدان أو اختلاف في الأبعاد.
2. **تماثل السمات (feature parity):** التأكد من أنّ السمات المستخرجة في المسار الحي مطابقة لتلك المستخرجة في التدريب للجولة نفسها. النتيجة: تطابق مضمون باستيراد المستخرج نفسه على مستوى الشيفرة.
3. **الجولة الحية كاملةً (round-trip):** إطلاق هجوم حقيقي ← اكتشافه وتصنيفه في live_runs.py ← كتابة الوثيقة في ml-predictions ← قراءتها وعرضها في Kibana. النتيجة: صُفّ المسح بثقة ٩٩,٢٪ وحُزّنت وثيقته بحقولها الصحيحة وظهرت في اللوحة.
4. **تكامل عبر البيئات:** التأكد من أنّ النموذج المحفوظ على Windows يُحمّل ويعمل على Kali. النتيجة: نجح بعد مطابقة إصدارات المكتبات (xgboost 3.2.0، sklearn 1.9.0).

٤.٣ اختبارات الأمان (security tests)

تشمل أمن النشر نفسه وفعالية الكشف الأمني الذي يقدمه النظام:

1. **عزل البيئة:** التحقق من أنّ حركة الهجوم لا تتسرّب من المختبر المعزول إلى أيّ شبكة حقيقية، إذ ينحصر كلّ شيء ضمن الجهاز الواحد. النتيجة: عزل مؤكّد، ويؤكّده ثبات العنوان المصدري 192.168.32.1 داخل حدود الجهاز.
2. **التحكّم بالوصول إلى Elasticsearch:** التأكد من أنّ Elasticsearch مقيدّ بالعنوان المحلي ولا يُوصّل إليه إلّا عبر نفق SSH. النتيجة: رُفِض الوصول المباشر من الخارج، واقتصصر على النفق.

3. **نظافة الأسرار:** التحقق من عدم رفع أي مفاتيح أو اعتمادات حقيقية إلى المستودع (تستبعد قواعدها قواعد .gitignore)، وأن كلمة المرور الوحيدة هي كلمة مزيفة متعمدة في الـ honeypot. النتيجة: لا أسرار في المستودع.
4. **فعالية الكشف الأمني:** تُعامل فئات الهجوم الخمس بوصفها حالات اختبار أمنية ضد الـ honeypot؛ وقد كشف الاختبار أن التصنيف القائم على القواعد في Tanner يرصد XSS فقط ويعدّ حقن SQL وإدراج الملفات وحقن الأوامر حميدةً — وهي ثغرة كشف حقيقية. وفي المقابل، بلغ المصنّف القائم على التعلّم دقّة ٩٨,٦٧٪ على الفئات الخمس، فغطّى ما تغط به القواعد.
5. **سلامة النموذج من التسريب (leakage audit):** مراجعة السمات للتأكد من أن النموذج لا يعتمد على حقول تخصّ البيئة أو تكشف الهوية (العناوين، وأرقام المنافذ، ومعرّفات الجلسات، ووقت اليوم، وحقل تصنيف Tanner). النتيجة: مجموعة سمات آمنة من التسريب، ويؤكّدها تطابق المجموعة السلوكية مع الكاملة (النموذج يصنّف بالسلوك لا بالمستشعر).

٤.٤ اختبار التحميل (load test)

يستهدف سلوك النظام واستقراره تحت حملٍ عالٍ، وهو ما تحقّقه فئة حجب الخدمة أصلاً:

1. **توليد الحمل:** إخضاع الـ honeypot لفيضاناتٍ عبر hping3 (فيضانات SYN وACK وFIN وXMAS وRST وUDP)، وab وsiege (فيضانات HTTP)، وslowhttptest (هجمات الحجب البطيئة). وضبطت المعدلات بعناية (فواصل زمنية وحدود للعدد والمدة) كي لا تنهار الآلة الافتراضية نفسها — وهو قيدٌ متعمّد في تصميم الاختبار. النتيجة: الثّقُطت الفيضانات وبقي النظام مستقرّاً (بلغت بعض الجولات نحو 90 ألف حدث).
2. **سلوك التسجيل تحت الحمل:** كشف الاختبار أن Suricata يسجّل بعض أحداث التدفّق متأخّرةً ٦٠-١١٠ ثانية تحت فيضانات TCP نصف المفتوحة، وقد تم حل ذلك بفترات تصريف (drain) داخل نوافذ الجمع وبفجوة حملٍ أطول (١٢٠ ثانية) للتصنيف الحي لحجب الخدمة.
3. **حمل خطّ المعالجة:** التأكّد من أن خطّ التصدير واستخراج السمات يتعامل مع حجم بياناتٍ كبير (نحو ١,٤ مليون حدث) دون إخفاق، بفضل ترقيم صفحات Scroll API وإعادة المحاولة عند التعرّض. النتيجة: اكتملت المعالجة كاملةً.
4. **المصنّف الحي تحت فيضانٍ متواصل:** التحقق من أن الحدّ الأقصى لطول الجولة (max_run) يفرض التصنيف للفيضانات التي لا تهدأ. النتيجة: صنّف فيضان حجب خدمةٍ متواصل بثقة ٩٩,٢٪.

٩. تحليل المهاجم: المصدر وطريقة الدخول والهدف

يهتم محل الأمن بثلاثة أسئلة كلاسيكية عن أي هجوم: من المهاجم (المصدر)، وكيف دخل (طريقة الدخول)، وما الذي يريده (الهدف). وما سجلته الـ honeypots يحمل الأدلة التي تُجيب عنها. ونضع بدايةً إطاراً صادقاً: في هذا المختبر المعزول المهاجم هو جهازنا (Kali) نفسه الذي يصل بالعنوان 192.168.32.1، فليست «من» مسألة حقيقية هنا، بل نعرض الحقول التي كانت ستُعرف مهاجماً حقيقياً لو كان النشر فعلياً.

٩.١ من هو المصدر

يُعرف المصدر بعنوانه src_ip ومنفذه src_port، لكنّ الأعرق منهما البصمات السلبية: يستنتج p0f نظام تشغيل المصدر من خصائص حزم TCP، ويلتقط Fatt و Cowrie بصمة عميل SSH (نسخته وترويسته)، وهي كثيراً ما تكشف الأداة المهاجمة (Hydra مقابل عميل OpenSSH عادي). كما يلحق T-Pot الموقع الجغرافي (GeoIP) بالعناوين الحقيقية. ونقرأ هذه الحقول في Discover على logstash-* بمرشح src_ip، بالنظر إلى src_ip و src_port و p0f.os وحقول عميل SSH. ونكرّر أنّ المصدر في مختبرنا ثابت دائماً (192.168.32.1)، فالقيمة هنا في الطريقة لا في هوية حقيقية.

```
"os.keyword": [
  | "Linux 2.2.x-3.x"
  ],|
```

الشكل 32: حقل os.keyword في بيانات p0f يظهر بصمة نظام تشغيل المصدر (Linux 2.2.x-3.x).

```
"src_ip": [
  | "192.168.32.1"
  ]
```

الشكل 33: حقل src_ip يظهر العنوان المصدري 192.168.32.1 في كلّ الأحداث.

☐ @timestamp	☐ dist	☐ src_ip	☐ os
☐ Aug 5, 2026 @ 18:49:28.000	1	192.168.32.1	Linux 2.2.x-3.x
☐ Aug 5, 2026 @ 18:49:28.000	1	192.168.32.1	Linux 2.2.x-3.x
☐ Aug 5, 2026 @ 18:49:28.000	1	192.168.32.1	Linux 2.2.x-3.x

الشكل 34: أحداث p0f في Kibana. حقلها @timestamp و dist و src_ip.

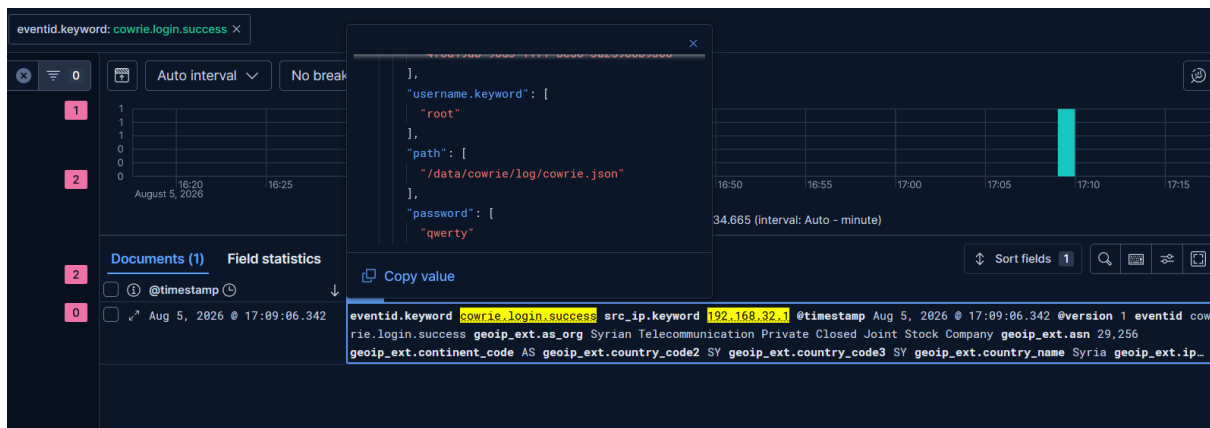
وفي cowrie:

Src IP - Top 10 - Dynamic	
Source IP	Count
192.168.32.1	2,242

الشكل 35: تسجيل Cowrie للعنوان المصدري 192.168.32.1

٩.٢ كيف تمّ الدخول

يُقرأ «كيف» من أمرين: الخدمة والمنفذ المستهدفان (dest_port وأيّ honeypot سجّل الحدث يحدّدان سطح الهجوم)، ومحاولات الدخول ونتيجتها. يسجّل Cowrie لكلّ محاولة حدث cowrie.login.failed أو cowrie.login.success يحمل username وpassword، وحدث النجاح هو لحظة الدخول وطريقته نفسها. ومن جولتنا الفعلية مثلاً واضح على تسلسل «فشل ثمّ نجاح»: جُربَت letmein ففشلت، ثمّ toor فنجحت، فسُجِّل cowrie.login.success بـ username=root وpassword=qwerty ورسالة «login attempt succeeded». وبذلك نستطيع أن نقول بدقّة كيف تمّ الوصول: تخمين عبر الإنترنت على SSH نجحت على root:qwerty بعد سلسلة إخفاقات. ويربط الحقل session هذا الدخول بكلّ ما يليه من نشاط.



الشكل 36: حدث cowrie.login.success يُظهر الزوج الناجح root/qwerty.

٩.٣ ما هدف المهاجم

يُقرأ الهدف مباشرةً من السلوك داخل الجلسة، وهو أغنى ما في بيانات ما بعد الاختراق. يسجّل Cowrie كلّ أمر يُدخّله المهاجم في cowrie.command.input، وتسلسل الأوامر هو النية نفسها. ومن أوامرنا الفعلية الأكثر تكراراً: whoami وid وuname وcat /etc/passwd وhostname وnetstat وps aux وwget http://example.com/setup.sh (وهي استطلاع)، ثمّ (تنزيل حمولة ومحاولة

بقاء)، وأحياناً `cat /etc/shadow` و `cat /root/.ssh/authorized_keys` و `crontab -l` (تصعيد وبقاء). ويقدم Cowrie نفسه جهاز تخزين QNAP (يظهر سطر الصدفة #~: root@NAS326) ويردّ رداً واقعية (فـ `whoami` يعيد `root`، و `uname -a` يعيد نظام Ubuntu 22.04). وتُسجّل أوامر `curl/wget` بوصفها `cowrie.command.input` فتلتقط نية التنزيل. أما في هجوم الويب فتكشف الحمولة الهدف: حقن SQL مثل `' OR '1'='1` (سرقة بيانات)، وإدراج ملفات مثل `etc/passwd` مثل `/?page=../../../../etc/passwd`، وحقن أوامر مثل `/?cmd=cat /etc/passwd` وفي حجب الخدمة يكشف الحجم والمعدل على منفذ واحد أنّ الهدف تعطيل لا دخول.

Src IP - Top 10 - Dynamic		Cowrie Input - Top 10	
Source IP	Count	Command Line Input	Count
192.168.32.1	729	cat /etc/passwd	1
		exit	1
		ifconfig	1
		ls	1
		ps	1
		uname -a	1
		whoami	1

الشكل 37: تسلسل أوامر `cowrie.command.input` داخل إحدى جلسات ما بعد الاختراق.

٩.٤ ربط الأسئلة الثلاثة بسمات النموذج

تنطبق الأسئلة الثلاثة على عائلات السمات:

الجدول 7: ربط أسئلة التحليل الثلاثة بمصادر الدليل وسمات النموذج المقابلة.

السؤال	مصدر الدليل	السمات المقابلة
من (المصدر)	<code>p0f</code> ، <code>src_ip</code> ، بصمة عميل SSH	سمات المنافذ والبروتوكولات، والرؤية الشبكية
كيف (الدخول)	<code>cowrie.login.*</code> ، <code>dest_port</code>	<code>'n_login_fail</code> <code>'n_login_success</code> <code>any_login_success</code>
ماذا (الهدف)	<code>'cowrie.command.input</code> روابط التنزيل، حمولات الويب، معدل الفيضان	<code>'n_commands</code> <code>distinct_commands</code> ، وسمات الويب والتوقيت

أي أنّ البيانات نفسها التي تُجيب عن «من/كيف/ماذا».

١٠. ملاحظات على قصور الـ honeypots

أثناء تنفيذ الحملات، رصدنا عدداً من أوجه القصور والسلوكيات الغريبة في أدوات الـ honeypot نفسها، بعضها يصلح بصمة كاشفة (fingerprint) لها، وأحدها يبرّر جوهر توجّهنا نحو التعلّم الآلي. نوجزها فيما يلي:

1. **Tanner يصنّف XSS فقط:** عند إرسال حمولاتٍ متنوّعة، سجّل Tanner حمولة XSS على أنّها xss، بينما وسّم كلّ ما عداها — حقن SQL وإدراج الملفات وحقن الأوامر — على أنّه index (أي طلبٌ حميد). فهو يكشف XSS بموثوقية، لكنّه يخطئ في تصنيف بقية الهجمات فيعدّها سليمة. وهذا بالضبط ما يبرّر اللجوء إلى التعلّم الآلي: فالكشف القائم على القواعد (rule-based) يُفَلِّت أغلب الهجمات، بينما يتعلّم نموذجنا تمييزها من سلوكها. ولاحظنا كذلك أنّ Nikto أقلّ فاعليّة بكثير ضدّ Tanner لأنّه «يعيد الرمز 200 لكلّ طلب».

1. **خلل تنزيل الملفات في Cowrie:** ينتهي أمر wget داخل Cowrie بخطأ برمجيّ (KeyError 's463')، فلا تُسجّل عمليات التنزيل إطلاقاً. كما لاحظنا سلوكاً مصطنعاً آخر: إعادة توجيه الكتابة إلى ملف (echo > file) تُسجّل بوصفها حدث «تنزيل ملف» بعنوانٍ فارغ — وهو أثرٌ من صنع الـ honeypot لا سلوك تنزيلٍ حقيقي. ولهذا السبب أسقطنا سمّي تنزيل الملفات من مجموعة البيانات (لثباتهما وعدم موثوقيتهما).

1. **بصمة RDP honeypot:** يعيد هذا الـ honeypot ردّاً ملتبساً مفاده أنّ «الحساب قد يكون صحيحاً لكنّه غير مفعل لسطح المكتب البعيد»، وهو ردٌّ نمطيّ يكشف أنّ الجهاز honeypot لا نظامٌ حقيقي.

1. **خلل ترويسة الخادم في Wordpot:** يُصدر Wordpot ترويسة (Server: ARRAY(0x...))، وهي خطأ برمجيّ يمرّر مرجع كائن/مصفوفة داخلياً إلى ترويسة Server بدل تحويله إلى نصٍّ أولاً. وتصلح هذه الترويسة المشوّهة بصمةً سلبية: من يراها يستنتج أنّه أمام Wordpot لا موقع WordPress حقيقي. وإلى جانب ما سبق، فإنّ التسجيل المتأخّر لأحداث تدفّق Suricata (المشار إليه في القسم الرابع) هو نفسه ملاحظة عمليّة على سلوك المنظومة، وقد شكّل تصميم نوافذ حجب الخدمة وفجوة الخمول في المصنّف الحيّ.

خلاصة

أنجزنا في هذا الشقّ العملي خطأً متكاملًا (pipeline) يبدأ من مختبر honeypot معزول، ويمرّ بجمع ١٥٠ جولة هجومٍ موزّعة على خمس فئات، فاستخراج ٣٢ سمة سلوكية، فتدريب سّة نماذج ومقارنتها، ثمّ نشر نموذج XGBoost وتشغيله في مصنّفٍ حيّ، وينتهي بلوحة معلومات Kibana تعرض تصنيفاته حيّاً وتُخزّنها بشكل دائم. والنتيجة منظومة تحقّق الفكرة المحورية للمشروع: **honeypot جاهز للنشر والتوظيف مدمجّ به نموذج ذكاء اصطناعي يصنّف نوع الهجوم آلياً**، بلغت دقّته نحو ٩٨,٧٪. ومن خصائص هذه المنظومة أنّ تصنيفها يقوم على سلوك الهجوم لا على المستشعر الذي رصده، مع انحصار الأخطاء كلّها في حدّ ضبابيّ واحد بين هجوم الويب وحجب الخدمة يُظهره الرسم نفسه بصدق. وبهذا يكتمل النظام: من توليد البيانات، إلى تنظيفها، إلى التصنيف، إلى عرضه الحيّ الموثق.